AI RECORD

TASK 1

From: https://chatgpt.com/c/6a16a08a-abe0-83ec-960a-d6ecc11bfe68

you asked: What kinds

What kinds of anomalies are likely in dirty_data.csv based on this brief?

chatgpt response: Based on t

Based on the assessment brief ***and a quick look at your*** *`Group004_dirty_data(1).csv`, the anomalies in `dirty_data.csv` are likely to be* ***rule-breaking values in columns other than*** *`order_id`,* ***`time`, quantities in*** *`order_items`,* ***and*** *`delivery_fee`* *(those four are explicitly error-free). Also:* ***max one anomaly per row****, and* ***each anomaly has exactly one valid fix****.*

Here's the shortlist I'd expect you to check first, roughly in order:

1. `date` anomalies

Possible issues:

- *invalid date format (not `YYYY-MM-DD`)*
- *wrong date entered*
- *weekend/weekday inconsistency relative to delivery fee model*

Why it matters:

- *delivery fee depends on* ***weekend vs weekday****, so a wrong date changes fee logic.*

Check:

```
python
pd.to_datetime(df['date'], errors='coerce')
```

2. `order_type` mismatching `time`

Very likely.

From the brief:

- *Breakfast = 08:00:00–12:00:00*
- *Lunch = 12:00:01–16:00:00*
- *Dinner = 16:00:01–20:00:00*

In your file there are definitely rows like:

- *`15:26:11` + `"Breakfast"` → should be Lunch*
- *`19:39:43` + `"Lunch"` → should be Dinner*

So this is a real anomaly in your CSV.

Check:

```
python
def expected_meal(t):
    ...
```

3. `branch_code` wrong

Likely:

- *wrong branch (`BK / NS / TP`)*
- *typo / swapped branch*

Why:

- *branch affects:*
  - *branch coordinates*
  - *shortest path*
  - *delivery fee formula*

Check against `branches.csv`.

4. `order_items` inconsistent with meal type

The brief says:

> *each meal has a distinct set of items; breakfast items cannot appear during lunch/dinner.*

Likely:

- *lunch item inside breakfast order*
- *typo in item name*
- *wrong meal label inferred from item set*

Approach:

- *infer menu groups from clean majority*
- *flag rows violating meal menu*

5. `order_price` incorrect

Very common in this assignment.

Brief hints:

use numpy.linalg to solve item unit prices.

Likely:

- *order total doesn't equal sum(items × unit price)*

Approach:

1. *derive unit prices from clean equations*
2. *recompute totals*
3. *compare*

Example:

```
python
abs(calc_total - row['order_price']) > 1e-6
```

6. `customer_lat` / `customer_lon` swapped or wrong

Possible:

- *coordinate typo*
- *lat/lon from wrong node*

Check against `nodes.csv`.

Melbourne sanity:

- *latitude ≈ `-37.x`*
- *longitude ≈ `144.x`*

Red flags:

- *lat positive*
- *lon negative*
- *exact coordinate not in nodes table*

7. `customerHasloyalty?` incorrect

Should only be:

- *`0`*
- *`1`*

Also can be validated against fee:

Because:

loyalty customers get 50% off delivery fee.

If fee matches only after discount: → loyalty flag may be wrong.

8. `distance_to_customer_KM` incorrect

Likely:

- *wrong shortest-path distance*
- *wrong node mapping*

Check:

1. *map branch → nearest node*
2. *customer coords → node*
3. *Dijkstra on edges*
4. *compare km*

What is NOT dirty (don't waste time)

Per brief: ✅ `order_id` ✅ `time` ✅ numeric quantity inside `order_items` ✅ `delivery_fee`

Most likely anomaly frequency in your file

Based on the brief + quick scan:

High probability:

1. *`order_type` ↔ `time`*
2. *`order_price`*
3. *`branch_code`*
4. *`order_items`*

Medium: 5. coordinates 6. distance

Lower: 7. date 8. loyalty flag

A practical workflow:

```
text
1. validate time vs order_type
2. infer menus
3. solve unit prices
4. validate order_price
5. validate coordinates
6. compute graph distance
7. validate fee/loyalty consistency
```

That order usually gives you the dependencies you need to uniquely fix each row.

you asked: The brief

The brief says each dirty row has at most one anomaly. How should we use that when validating rows?

*chatgpt response: That "**at*

*That "**at most one anomaly per row**" rule is actually one of the strongest constraints in the whole task. Use it as a **decision rule** when validations disagree.*

Here's the practical way to apply it.

Core idea

For each row:

1. *Run **all validation checks***
2. *Collect every field that appears invalid*
3. *If **0 fail** → row is clean*
4. *If **1 fails** → fix that field*
5. *If **2+ fail → don't fix both***
    - *assume **one field is wrong***
    - *ask: which single correction would make the other checks pass?*

That last part is where the constraint helps.

Example 1 — `order_type` vs `order_items`

Row:

```
text
time = 09:15:00
order_type = Lunch
order_items = [('Pancakes',2)]
```

Checks:

- *time says **Breakfast***
- *items also look **Breakfast***

Two apparent mismatches:

- *`order_type` mismatches time*
- *`order_type` mismatches items*

*Because **only one anomaly allowed**:*

✅ *fix* `order_type` → *Breakfast*

Do ***not*** *touch time or items.*

That single fix resolves everything.

Example 2 — `order_price` vs items

Row:

```
text
order_items = [('Burger',2)]
order_price = 14.50
```

Checks:

- *item names valid*
- *quantities valid*
- *calculated price = 29.00*

Could be:

- *wrong item?*
- *wrong price?*

But one-anomaly rule:

If changing `order_price` → *29.00 makes row consistent:*

✅ *only fix* `order_price`

Example 3 — branch / distance / coordinates

Row:

```
text
branch_code = BK
customer coords = valid node
distance = 8.2
```

Graph says:

- *BK → customer = 8.2* ❌
- *NS → customer = 8.2* ✅

Looks like:

- *branch wrong OR*
- *distance wrong*

*Since distance column is guaranteed clean in dirty data? No — **delivery_fee** is guaranteed clean; distance is not.*

So compare:

If changing branch makes everything consistent:

- *branch coords*
- *shortest path*
- *fee model*

✅ *fix branch only*

Else fix distance.

Best workflow: "single-fix scoring"

For rows with multiple failed checks:

For each suspicious field:

- *temporarily correct only that field*
- *rerun row validation*
- *count remaining failures*

Example:

```python
candidates = ['order_type','branch_code','order_price']

for col in candidates:
    temp = row.copy()
    temp[col] = inferred_fix[col]
    score = validate(temp)   # number of failed checks
```

Choose:

```text
lowest score
```

Ideally:

```text
one fix -> 0 failures
```

That should be the correct answer because:

all anomalies have one and only one possible fix.

Very useful interpretation

*Sometimes **multiple validators fail because of one bad column**.*

Example:

Wrong `branch_code` can cause:

- *branch mismatch*
- *distance mismatch*
- *fee-model mismatch*

*That is still **one anomaly**.*

So think:

> *"Which one field explains all downstream failures?"*

rather than

> *"How many checks failed?"*

A clean row-level checklist

For each row:

```text
□ validate date
□ validate order_type vs time
□ validate order_items vs meal menu
□ validate order_price
□ validate coords
□ validate distance
□ validate loyalty/fee consistency
```

Then:

```text
0 failed -> keep
1 failed -> fix that field
2+ failed ->
    test one-field fixes
    choose the single fix that resolves all
```

That constraint massively reduces ambiguity—use it aggressively.

you asked: How should

How should I validate breakfast, lunch and dinner order types using time?

chatgpt response: For this c

*For this column, validation is straightforward because the brief gives **exact time windows** and says `time` itself is error-free. That means **use** `time` **as the source of truth** and check whether `order_type` matches it.*

Here's the mapping:

Time range	*Expected `order_type`*
`08:00:00` → `12:00:00`	*Breakfast*
`12:00:01` → `16:00:00`	*Lunch*
`16:00:01` → `20:00:00`	*Dinner*

Important edge cases:

- *`12:00:00` = **Breakfast***
- *`12:00:01` = **Lunch***
- *`16:00:00` = **Lunch***
- *`16:00:01` = **Dinner***

Clean pandas validation

```python
import pandas as pd

# convert once
df["time_dt"] = pd.to_datetime(df["time"], format="%H:%M:%S").dt.time

def expected_order_type(t):
    if pd.Timestamp("08:00:00").time() <= t <= pd.Timestamp("12:00:00").time():
        return "Breakfast"

    elif pd.Timestamp("12:00:01").time() <= t <= pd.Timestamp("16:00:00").time():
        return "Lunch"

    elif pd.Timestamp("16:00:01").time() <= t <= pd.Timestamp("20:00:00").time():
```

```
        return "Dinner"

    else:
        return None   # outside service hours

df["expected_order_type"] = df["time_dt"].apply(expected_order_type)

# mismatches
wrong_order_type = df[
    df["order_type"] != df["expected_order_type"]
]

print(wrong_order_type[
    ["time", "order_type", "expected_order_type"]
])
```

Example

Row:

```
text
time = 09:12:33
order_type = Lunch
```

Validation:

- *09:12:33 falls in breakfast*

Expected:

```
text
Breakfast
```

Mismatch → fix order_type.

Another:

```
text
time = 16:00:00
order_type = Lunch
```

*That is **correct**.*

But:

```text
time = 16:00:01
order_type = Lunch
```

Should be:

```text
Dinner
```

How to use with "one anomaly per row"

If you see:

```text
time = 10:45:00
order_type = Dinner
order_items = breakfast-looking items
```

Then:

- *time says Breakfast*
- *items say Breakfast*

Only `order_type` is inconsistent

✅ *fix `order_type = Breakfast`*

Because `time` is guaranteed clean and each row has max one anomaly.

So in practice:

```text
time → expected meal
compare to order_type
if mismatch → order_type is the anomaly
```

you asked: How should

How should I validate breakfast, lunch and dinner order types using time?

chatgpt response: Use the `t

*Use the `time` column as the **ground truth**. The brief explicitly says `time` is error-free, and meal windows are fixed. So validation is just:*

Given a row's time, what meal should it be? Then compare that to order_type.

From the brief:

Time	*Expected* `order_type`
`08:00:00` → `12:00:00`	*Breakfast*
`12:00:01` → `16:00:00`	*Lunch*
`16:00:01` → `20:00:00`	*Dinner*

Important boundary cases

These matter a lot:

```text
08:00:00  -> Breakfast
12:00:00  -> Breakfast
12:00:01  -> Lunch
16:00:00  -> Lunch
16:00:01  -> Dinner
20:00:00  -> Dinner
```

Pandas validation

```python
import pandas as pd
from datetime import time

# parse once
df["parsed_time"] = pd.to_datetime(
    df["time"],
    format="%H:%M:%S"
).dt.time



def expected_order_type(t):
    if time(8, 0, 0) <= t <= time(12, 0, 0):
        return "Breakfast"

    elif time(12, 0, 1) <= t <= time(16, 0, 0):
        return "Lunch"
```

```
    elif time(16, 0, 1) <= t <= time(20, 0, 0):
        return "Dinner"

    else:
        return None   # outside valid service hours


df["expected_order_type"] = (
    df["parsed_time"]
    .apply(expected_order_type)
)

# mismatches
wrong = df[
    df["order_type"] != df["expected_order_type"]
]

print(
    wrong[
        ["time", "order_type", "expected_order_type"]
    ]
)
```

Example

Row:

```
text
 time = 09:42:18
 order_type = Lunch
```

Validation:

```
text
 09:42:18 -> Breakfast
 Lunch != Breakfast
```

→ *anomaly is:*

```text
order_type = Breakfast
```

Another:

```text
time = 16:00:00
order_type = Lunch
```

*That is **correct**.*

But:

```text
time = 16:00:01
order_type = Lunch
```

Should be:

```text
Dinner
```

With the "one anomaly per row" rule

Because `time` is guaranteed correct:

If row says:

```text
time = 10:15:00
order_type = Dinner
```

Then:

```text
expected = Breakfast
```

Fix:

```text
order_type = Breakfast
```

You don't need to question the time column.

So the workflow is:

```text
time
→ map to expected meal
→ compare with order_type
→ if mismatch: fix order_type
```

you asked: What's a c

What's a clean way to infer item prices from valid orders using numpy?

chatgpt response: A clean wa

A clean way is:

1. ***Pick rows you trust*** *(or rows where* `order_price` *is consistent with everything else)*
2. *Convert* `order_items` *into an* ***equation***
3. *Stack equations into matrix* ***A***
4. *Solve* ***Ax = b*** *with NumPy*
5. *Verify against held-out rows*

That matches the brief's hint to use `numpy.linalg`*.*

Example.

1. Parse `order_items`

Say rows look like:

```text
[('Pancakes',2), ('Coffee',1)]
```

Convert to Python:

```python
import ast


df["parsed_items"] = df["order_items"].apply(ast.literal_eval)
```

2. Infer the unique menu items

Example:

```text
Pancakes
Coffee
Burger
Fries
```

Code:

```python
all_items = sorted({
    item
    for row in df["parsed_items"]
    for item, qty in row
})

item_index = {
    item: i
    for i, item in enumerate(all_items)
}
```

Now each item becomes a column.

3. Build equation matrix

Example row:

```text
[('Pancakes',2), ('Coffee',1)]
order_price = 23
```

becomes:

```text
2*Pancakes + 1*Coffee = 23
```

Code:

```python
import numpy as np

A = []
b = []
```

```
for _, row in df.iterrows():
    vec = np.zeros(len(all_items))

    for item, qty in row["parsed_items"]:
        vec[item_index[item]] = qty

    A.append(vec)
    b.append(row["order_price"])

A = np.array(A)
b = np.array(b)
```

4. Solve with NumPy

If exactly determined:

```
python
prices = np.linalg.solve(A, b)
```

Usually you'll have more rows than items → better:

```
python
prices, residuals, rank, s = np.linalg.lstsq(
    A,
    b,
    rcond=None
)
```

Map back:

```
python
price_lookup = {
    item: round(prices[i], 2)
    for item, i in item_index.items()
}

print(price_lookup)
```

Example output:

```text
{
 'Coffee': 4.50,
 'Pancakes': 9.00,
 'Burger': 14.00
}
```

5. Validate recovered prices

Recalculate totals:

```python
def calc_total(items):
    return round(
        sum(
            price_lookup[item] * qty
            for item, qty in items
        ),
        2
    )

df["recalc"] = df["parsed_items"].apply(calc_total)

df["matches"] = (
    abs(df["recalc"] - df["order_price"])
    < 0.01
)
```

High match rate = good.

For your assignment: do it **per meal type**

Because:

> breakfast / lunch / dinner have distinct menus.

Best workflow:

```text
Breakfast rows -> solve breakfast prices
Lunch rows     -> solve lunch prices
Dinner rows    -> solve dinner prices
```

Why:

- *fewer unknowns*
- *cleaner equations*
- *easier debugging*

Example:

```
python
for meal in ["Breakfast", "Lunch", "Dinner"]:
    subset = df[df["order_type"] == meal]
    ...
```

Also: only use trustworthy rows

Avoid rows where:

- `order_type` *already looks wrong*
- *item names look suspicious*
- *branch uncertainty affects validation*

Good candidates:

- *rows passing:*
  - *time ↔ order_type*
  - *item names valid*

Then solve prices from those.

A nice rule:

```
text
use many rows
solve prices
recalculate
rows with large residuals are likely order_price anomalies
```

That works really well with the "one anomaly per row" constraint.

you asked: How can I

How can I check whether order_price matches order_items after prices are inferred?

chatgpt response: Once you'v

Once you've inferred item prices, the check is simple:

Recalculate the total from order_items → compare it to order_price.

If they differ beyond a tiny tolerance, that row is likely an `order_price` anomaly.

Here's a clean workflow.

1. Assume you already have prices

Example:

```python
price_lookup = {
    "Pancakes": 12.00,
    "Coffee": 4.50,
    "Burger": 15.00,
    "Fries": 6.00
}
```

2. Parse `order_items`

Your CSV stores text like:

```text
[('Pancakes', 2), ('Coffee', 1)]
```

Convert:

```python
import ast

df["parsed_items"] = (
    df["order_items"]
    .apply(ast.literal_eval)
)
```

3. Recalculate each row

```python
def calc_order_total(items):
    total = 0

    for item, qty in items:
        total += price_lookup[item] * qty
```

```
    return round(total, 2)


df["calculated_price"] = (
    df["parsed_items"]
    .apply(calc_order_total)
)
```

Example:

```
text
[('Pancakes',2), ('Coffee',1)]

2*12 + 1*4.5 = 28.50
```

4. Compare with tolerance

Use a tolerance for floating point:

```
python
TOL = 0.01

df["price_matches"] = (
    abs(
        df["calculated_price"]
        - df["order_price"]
    ) <= TOL
)
```

5. Show suspicious rows

```
python
bad_price_rows = df[
    ~df["price_matches"]
]

print(
    bad_price_rows[
        [
            "order_id",
```

```
        "order_items",
        "order_price",
        "calculated_price"
      ]
    ]
)
```

Example output:

```
text
order_id   order_items                order_price   calculated_price
A104       [('Burger',2)]             20.00         30.00
A117       [('Coffee',1)]             4.50          4.50
```

Only row A104 is suspicious.

6. Use the "one anomaly per row" rule

Important.

If row shows:

```
text
order_items            [('Burger',2)]
calculated_price  30.00
order_price            20.00
```

and:

- meal type valid
- item names valid
- quantities guaranteed clean

Then:

✅ anomaly = `order_price`

Fix:

```
text
20.00 -> 30.00
```

Because quantities in `order_items` are guaranteed correct, and each dirty row has max one anomaly.

A useful debug column

This helps rank anomalies:

```python
df["price_diff"] = (
    df["calculated_price"]
    - df["order_price"]
).round(2)
```

Example:

```text
0.00   -> correct
+8.50  -> undercharged
-4.50  -> overcharged
```

Very handy when reviewing.

Best practice: validate by meal

Since breakfast/lunch/dinner have different menus:

```python
for meal in ["Breakfast", "Lunch", "Dinner"]:
    subset = df[df["order_type"] == meal]
```

Use that meal's `price_lookup`*.*

That avoids cross-meal item confusion and makes mismatches much clearer.

you asked: How do I v

How do I verify that an item belongs to the correct meal menu?

chatgpt response: A reliable

A reliable way is to ***infer each meal's menu from rows that are already trustworthy****, then validate every row against those inferred menus.*

Because the brief says:

- *Breakfast / Lunch / Dinner each have* ***distinct items***
- *an item cannot appear in multiple meals*
- `time` *is clean*

That means you can use `time → expected meal` as ground truth and learn the menu from there.

Step 1 — assign expected meal from time

```python
from datetime import time
import pandas as pd

df["parsed_time"] = pd.to_datetime(
    df["time"],
    format="%H:%M:%S"
).dt.time



def expected_meal(t):
    if time(8,0,0) <= t <= time(12,0,0):
        return "Breakfast"

    elif time(12,0,1) <= t <= time(16,0,0):
        return "Lunch"

    elif time(16,0,1) <= t <= time(20,0,0):
        return "Dinner"



df["expected_meal"] = (
    df["parsed_time"]
    .apply(expected_meal)
)
```

Now every row has a trusted meal label.

Step 2 — parse items

```python
import ast

df["parsed_items"] = (
    df["order_items"]
```

```
    .apply(ast.literal_eval)
)
```

Step 3 — infer menu sets

Collect items seen during each trusted meal:

```python
meal_menus = {
    "Breakfast": set(),
    "Lunch": set(),
    "Dinner": set()
}

for _, row in df.iterrows():

    meal = row["expected_meal"]

    for item, qty in row["parsed_items"]:
        meal_menus[meal].add(item)

print(meal_menus)
```

Example:

```text
Breakfast:
{'Coffee','Pancakes','Toast'}

Lunch:
{'Burger','Fries','Salad'}

Dinner:
{'Steak','Pasta','Wine'}
```

Step 4 — find overlaps

Menus must be distinct.

Check:

```python
print(
    meal_menus["Breakfast"]
    & meal_menus["Lunch"]
)

print(
    meal_menus["Lunch"]
    & meal_menus["Dinner"]
)

print(
    meal_menus["Breakfast"]
    & meal_menus["Dinner"]
)
```

Expected:

```text
set()
set()
set()
```

If overlaps exist:

- item may be typo
- some rows dirty

Use frequency counts to resolve.

Step 5 — validate each row

For every row:

```python
def items_match_meal(row):

    meal = row["expected_meal"]
    allowed = meal_menus[meal]

    for item, qty in row["parsed_items"]:
        if item not in allowed:
```

```
            return False

    return True



df["items_valid_for_meal"] = (
    df.apply(items_match_meal, axis=1)
)
```

Flag bad rows:

```
python
bad = df[
    ~df["items_valid_for_meal"]
]

print(
    bad[
        [
            "order_id",
            "time",
            "order_items"
        ]
    ]
)
```

Example

Row:

```
text
time = 09:10:00
expected_meal = Breakfast
order_items = [('Burger',1)]
```

Breakfast menu:

```
text
{'Coffee','Toast','Pancakes'}
```

`Burger` not allowed

→ row fails.

Then use one-anomaly rule:

Could be:

- wrong `order_type`
- wrong item name

But since `time` is correct:

If item belongs only to lunch:

```text
Burger -> Lunch
```

then likely `order_items` anomaly.

Even stronger: item → meal map

Build reverse lookup:

```python
item_to_meal = {}

for meal, items in meal_menus.items():
    for item in items:
        item_to_meal[item] = meal
```

Then:

```python
for item, qty in row["parsed_items"]:
    print(item, item_to_meal[item])
```

Example:

```text
Coffee -> Breakfast
Burger -> Lunch
```

Very fast for debugging.

In short:

```text
time -> expected meal
```

```
expected meal -> inferred menu
each item must belong to that menu only
```

That's usually the cleanest validation approach for this assignment.

you asked: How can I

How can I calculate shortest-path distance using nodes and edges?

chatgpt response: For this a

For this assignment, the shortest-path distance is:

> *branch location → nearest graph node → Dijkstra through edges → customer node*

using:

- `branches.csv`
- `nodes.csv`
- `edges.csv`

*The cleanest way is with **NetworkX**.*

1. Load the files

```python
import pandas as pd
import networkx as nx

branches = pd.read_csv("branches.csv")
nodes = pd.read_csv("nodes.csv")
edges = pd.read_csv("edges.csv")
```

Your uploaded files look like:

branches

```text
NS  -37.773803  144.983647
TP  -37.861835  144.905716
BK  -37.815834  145.046450
```

edges

```text
u         v                  distance(m)
711327755  711332946  58
```

nodes

```text
node          lat         lon
711327755  -37.80...  144.95...
```

2. Build the road graph

Use `distance(m)` as weight.

```python
G = nx.Graph()

for _, row in edges.iterrows():

    G.add_edge(
        int(row["u"]),
        int(row["v"]),
        weight=float(row["distance(m)"])
    )
```

Now NetworkX knows the road network.

3. Map coordinates → nearest node

Branch/customer coordinates won't directly equal node IDs.

Find closest node:

```python
def nearest_node(lat, lon, nodes_df):

    dists = (
        (nodes_df["lat"] - lat)**2
        + (nodes_df["lon"] - lon)**2
    )

    idx = dists.idxmin()
```

```
    return int(nodes_df.loc[idx, "node"])
```

Example:

```python
branch_row = branches[
    branches["branch_code"] == "NS"
].iloc[0]

branch_node = nearest_node(
    branch_row["branch_lat"],
    branch_row["branch_lon"],
    nodes
)

print(branch_node)
```

4. Customer coordinates → node

For each dirty row:

```python
customer_node = nearest_node(
    row["customer_lat"],
    row["customer_lon"],
    nodes
)
```

5. Dijkstra shortest path

Compute metres:

```python
dist_m = nx.shortest_path_length(
    G,
    source=branch_node,
    target=customer_node,
    weight="weight"
)
```

Convert to km:

```python
dist_km = round(
    dist_m / 1000,
    3
)
```

That should match:

```text
distance_to_customer_KM
```

6. Validate a dataframe row

Example helper:

```python
def calc_distance_km(row):

    branch = branches[
        branches["branch_code"]
        == row["branch_code"]
    ].iloc[0]

    branch_node = nearest_node(
        branch["branch_lat"],
        branch["branch_lon"],
        nodes
    )

    customer_node = nearest_node(
        row["customer_lat"],
        row["customer_lon"],
        nodes
    )

    dist_m = nx.shortest_path_length(
        G,
        source=branch_node,
        target=customer_node,
        weight="weight"
```

```
    )

    return round(dist_m / 1000, 3)
```

Apply:

```python
df["calc_distance"] = (
    df.apply(calc_distance_km, axis=1)
)

df["distance_match"] = (
    abs(
        df["calc_distance"]
        - df["distance_to_customer_KM"]
    ) < 0.01
)
```

7. Using the “one anomaly per row” rule

If row says:

```text
branch = BK
coords = valid
distance = 4.812
```

but graph gives:

```text
BK -> 7.231
NS -> 4.812
```

Then likely:

✅ `branch_code` is wrong

not distance.

Or if branch and coords both align but graph differs:

✅ `distance_to_customer_KM` is wrong

That rule makes debugging much easier.

Small tip

Your `edges.csv` includes an `Unnamed: 0` column.

Ignore it:

```python
edges = edges.drop(
    columns=["Unnamed: 0"],
    errors="ignore"
)
```

Then build the graph.

Workflow:

```text
branch coords -> nearest node
customer coords -> nearest node
Dijkstra(weight=distance(m))
convert m -> km
compare to distance_to_customer_KM
```

That's the standard way your marker will expect.

you asked: How do I m

How do I map customer latitude and longitude to the nearest graph node?

chatgpt response: The cleane

The cleanest way is:

> *For each (customer_lat, customer_lon), find the node in nodes.csv with the smallest distance.*

*Because customer coordinates come from `nodes.csv` per the brief, the correct node should usually be an **exact match**. If floating-point formatting changed slightly, use nearest-neighbour.*

Here's a practical approach.

1. Exact match first (best)

Your `nodes.csv` looks like:

```text
node          lat         lon
711327755   -37.81     144.95
...
```

Build a lookup:

```python
nodes_lookup = {
    (row["lat"], row["lon"]): int(row["node"])
    for _, row in nodes.iterrows()
}
```

Then:

```python
def get_node_exact(lat, lon):
    return nodes_lookup.get((lat, lon))
```

Example:

```python
node_id = get_node_exact(
    row["customer_lat"],
    row["customer_lon"]
)
```

If found:

✅ done.

2. Nearest-node fallback

Sometimes CSV precision differs:

```text
-37.812345
```

vs

```text
-37.812344999
```

Use squared Euclidean distance:

```python
def nearest_node(lat, lon, nodes_df):

    d2 = (
        (nodes_df["lat"] - lat)**2
        + (nodes_df["lon"] - lon)**2
    )

    idx = d2.idxmin()

    return int(
        nodes_df.loc[idx, "node"]
    )
```

Example:

```python
customer_node = nearest_node(
    row["customer_lat"],
    row["customer_lon"],
    nodes
)
```

3. Best combined helper

Use exact first, then fallback:

```python
def customer_to_node(lat, lon, nodes_df):

    exact = nodes_df[
        (nodes_df["lat"] == lat)
        &
        (nodes_df["lon"] == lon)
    ]

    if not exact.empty:
        return int(
            exact.iloc[0]["node"]
        )
```

```
    # fallback
    d2 = (
        (nodes_df["lat"] - lat)**2
        + (nodes_df["lon"] - lon)**2
    )

    idx = d2.idxmin()

    return int(
        nodes_df.loc[idx, "node"]
    )
```

Then:

```python
df["customer_node"] = df.apply(
    lambda r: customer_to_node(
        r["customer_lat"],
        r["customer_lon"],
        nodes
    ),
    axis=1
)
```

4. Quick sanity check

Check mapped coordinates:

```python
node_id = customer_to_node(lat, lon, nodes)

matched = nodes[
    nodes["node"] == node_id
]

print(matched)
```

Should be nearly identical:

```text
customer_lat ≈ node.lat
customer_lon ≈ node.lon
```

5. Useful tolerance check

You can verify closeness:

```python
import math

def coord_gap(lat1, lon1, lat2, lon2):
    return math.sqrt(
        (lat1-lat2)**2
        +
        (lon1-lon2)**2
    )
```

Then:

```python
gap = coord_gap(
    row["customer_lat"],
    row["customer_lon"],
    matched.iloc[0]["lat"],
    matched.iloc[0]["lon"]
)
```

Expected:

```text
0
```

or extremely tiny.

In your assignment

Once mapped:

```text
customer lat/lon -> customer node
branch lat/lon   -> branch node
```

```
Dijkstra shortest path
compare with distance_to_customer_KM
```

And because there's max one anomaly per dirty row:

If:

- *customer coordinates map perfectly*
- *branch maps perfectly*
- *shortest path differs*

→ likely `distance_to_customer_KM` anomaly.

If coordinates don't map to any nearby node:

→ likely `customer_lat` / `customer_lon` anomaly.

you asked: How can I

How can I verify my calculated distances against the dataset?

chatgpt response: Once you've

Once you've calculated shortest-path distances, the check is:

> *compare your calc_distance_km with distance_to_customer_KM row by row*

Then flag rows where the difference is too large.

Here's a clean way.

1. Compute your distance

Assume you already have:

```
python
df["calc_distance_km"] = (
    df.apply(calc_distance_km, axis=1)
)
```

Example:

```
text
calc_distance_km
4.813
```

```
2.177
6.502
```

2. Difference column

Subtract dataset value:

```python
df["distance_diff"] = (
    df["calc_distance_km"]
    - df["distance_to_customer_KM"]
).round(3)
```

Example:

```text
4.813 - 4.813 = 0.000
2.177 - 2.176 = 0.001
6.502 - 8.900 = -2.398
```

Very easy to inspect.

3. Use tolerance

Because:

- *graph distance is in metres*
- *dataset stores km*
- *rounding may happen*

Don't compare exact equality.

Use tolerance:

```python
TOL = 0.01

df["distance_matches"] = (
    abs(df["distance_diff"])
    <= TOL
)
```

Meaning:

```text
<= 0.01 km
```

= within 10 metres.

Usually safe.

4. Show suspicious rows

```python
bad_distance = df[
    ~df["distance_matches"]
]

print(
    bad_distance[
        [
            "order_id",
            "branch_code",
            "customer_lat",
            "customer_lon",
            "distance_to_customer_KM",
            "calc_distance_km",
            "distance_diff"
        ]
    ]
)
```

Example:

```text
A113
dataset = 8.900
calc    = 6.502
diff    = -2.398
```

That row needs investigation.

5. Interpret with one-anomaly rule

A mismatch can come from:

- `branch_code`

- *customer_lat*
- *customer_lon*
- *distance_to_customer_KM*

*But only **one** anomaly per row.*

So:

Case A

Branch changed:

```
text
BK -> 8.9 ❌
NS -> 6.5 ✅
```

Then:

✅ *branch_code wrong*

Case B

Coords map to wrong node

Current:

```
text
customer node -> 8.9
```

Nearest neighbouring node:

```
text
6.5
```

Then:

✅ *customer coordinate anomaly*

Case C

Everything maps correctly but value differs

Then:

✅ *distance_to_customer_KM wrong*

6. Helpful summary

How many fail?

```python
print(
    (~df["distance_matches"]).sum()
)
```

Largest mismatches:

```python
print(
    df.sort_values(
        "distance_diff",
        key=abs,
        ascending=False
    ).head(10)
)
```

Very useful during cleaning.

Good validation rule

In practice:

```text
|difference| <= 0.01 km -> valid
|difference| > 0.01 km  -> investigate
```

Then use:

```text
branch?
coords?
or distance field?
```

*and pick the **single** fix that makes the row consistent.*

you asked: What's a g

What's a good way to model delivery_fee using distance and time-related features?

chatgpt response: A good app

A good approach for this assignment is exactly what the brief hints:

One linear regression per branch using:

That usually gives a very high R² if your training rows are clean.

1. Create features

The brief defines:

Weekend / weekday

```python
df["date_dt"] = pd.to_datetime(df["date"])

df["is_weekend"] = (
    df["date_dt"]
    .dt.dayofweek
    .isin([5, 6])
    .astype(int)
)
```

Result:

```text
weekday -> 0
weekend -> 1
```

Time of day

From the brief:

```text
morning   -> 0
afternoon -> 1
evening   -> 2
```

Code:

```python
from datetime import time

def time_code(t):

    t = pd.to_datetime(
        t,
```

```
        format="%H:%M:%S"
    ).time()

    if time(8,0,0) <= t <= time(12,0,0):
        return 0

    elif time(12,0,1) <= t <= time(16,0,0):
        return 1

    elif time(16,0,1) <= t <= time(20,0,0):
        return 2



df["time_code"] = (
    df["time"]
    .apply(time_code)
)
```

Distance

Already present:

```
python
df["distance_to_customer_KM"]
```

2. Remove loyalty discount first

Important.

Brief says:

> *loyalty customers get 50% off delivery fee.*

Undo discount before training:

```
python
def fee_before_discount(row):

    fee = row["delivery_fee"]

    if row["customerHasloyalty?"] == 1:
        return fee * 2
```

```
    return fee


df["base_fee"] = (
    df.apply(
        fee_before_discount,
        axis=1
    )
)
```

Now model the undiscounted fee.

3. Train one model per branch

Because the brief says:

> each branch calculates fee differently.

```python
from sklearn.linear_model import LinearRegression

models = {}

for branch in df["branch_code"].unique():

    sub = df[
        df["branch_code"] == branch
    ]

    X = sub[
        [
            "is_weekend",
            "time_code",
            "distance_to_customer_KM"
        ]
    ]

    y = sub["base_fee"]
```

```
    model = LinearRegression()
    model.fit(X, y)

    print(
        branch,
        model.score(X, y)
    )

    models[branch] = model
```

You want:

```
text
 R² > 0.95
```

and ideally very close to 1.

4. Predict and reapply loyalty

```
python
 def predict_fee(row):

    model = models[
        row["branch_code"]
    ]

    X = [[
        row["is_weekend"],
        row["time_code"],
        row["distance_to_customer_KM"]
    ]]

    fee = model.predict(X)[0]

    if row["customerHasloyalty?"] == 1:
        fee *= 0.5

    return round(fee, 2)
```

```
df["pred_fee"] = (
    df.apply(
        predict_fee,
        axis=1
    )
)
```

5. Compare with dataset

```
python
df["fee_diff"] = (
    df["pred_fee"]
    - df["delivery_fee"]
).round(2)

df["fee_match"] = (
    abs(df["fee_diff"])
    <= 0.01
)
```

Flag:

```
python
bad_fee = df[
    ~df["fee_match"]
]
```

6. Use it for anomaly detection

Example:

Dataset:

```
text
delivery_fee = 4.50
```

Model predicts:

```
text
4.50
```

Good.

But:

```
text
delivery_fee = 4.50
predicted = 9.00
```

If customer has loyalty:

```
text
9.00 * 0.5 = 4.50
```

Then:

✅ *loyalty flag is correct.*

If no loyalty:

→ *something else wrong.*

Best training rows

Use rows already validated for:

```
text
✓ order_type
✓ order_items
✓ order_price
✓ coordinates
✓ distance
```

Then fit the model.

That usually gives very clean coefficients and makes delivery-fee validation much easier.

you asked: Should loy

Should loyalty discount be reversed before fitting the fee model?

*chatgpt response: Yes — **re*

*Yes — **reverse the loyalty discount before fitting**.*

That's the cleanest and most accurate way for this assignment.

Why:

The brief says:

1. *delivery fee depends linearly on:*
    - *weekend/weekday*
    - *time of day*
    - *distance*
2. *then:*

    customers with loyalty get 50% discount

So mathematically:

```text
base_fee = linear(branch, weekend, time_code, distance)
```

then

```text
final_fee =
    base_fee * 0.5   if loyalty=1
    base_fee         if loyalty=0
```

If you train directly on the observed `delivery_fee`*, you mix:*

```text
4.00
8.00
```

for otherwise identical rows.

That makes the regression noisy and lowers R^2*.*

Correct workflow

Step 1 — undo discount

```python
def reverse_loyalty(row):

    fee = row["delivery_fee"]

    if row["customerHasloyalty?"] == 1:
        return fee * 2

    return fee
```

```
df["base_fee"] = (
    df.apply(
        reverse_loyalty,
        axis=1
    )
)
```

Example:

loyalty	*delivery_fee*	*base_fee*
0	*8.00*	*8.00*
1	*4.00*	*8.00*

Now both represent the same underlying fee.

Step 2 — fit model on `base_fee`

```
python
X = df[
    [
        "is_weekend",
        "time_code",
        "distance_to_customer_KM"
    ]
]

y = df["base_fee"]

model.fit(X, y)
```

Step 3 — predict

```
python
pred_base = model.predict(X)
```

Step 4 — apply discount again

```
python
def final_pred(row, pred_base):
```

```
    if row["customerHasloyalty?"] == 1:
        return round(pred_base * 0.5, 2)

    return round(pred_base, 2)
```

Then compare with dataset.

Quick sanity example

Suppose branch formula is:

```
text
fee = 2 + 0.5*time_code + 1.2*distance
```

Distance = 5 km Lunch = 1

Base fee:

```
text
2 + 0.5 + 6 = 8.5
```

Customer without loyalty:

```
text
8.50
```

Customer with loyalty:

```
text
4.25
```

If you train directly:

same features → two different targets

Bad.

If you reverse first:

both become:

```
text
8.50
```

Perfect linear fit.

Also useful for validation

If row predicts:

```
text
base_fee = 9.20
```

and loyalty = 1

Expected:

```
text
4.60
```

Dataset shows:

```
text
9.20
```

Then likely:

✅ *loyalty flag wrong*

or

```
text
customerHasloyalty? should be 1
```

So yes:

reverse loyalty before fitting, then reapply after prediction.

you asked: How do I d

How do I decide which rows are reliable enough for training?

chatgpt response: This is on

This is one of the key judgment calls in Task 1.

A practical rule is:

> *Train only on rows that pass all validations needed for that model and exclude anything suspicious.*

Because the brief wants a very high R^2 and dirty rows have at most one anomaly, even a few bad rows can distort the coefficients.

For `delivery_fee`, I'd build a ***trusted subset****.*

1. Start from columns that must be correct

The fee model depends on:

```
text
 date
 time
 branch_code
 customer_lat
 customer_lon
 distance_to_customer_KM
 customerHasloyalty?
 delivery_fee
```

But the brief guarantees:

✅ time ✅ delivery_fee

So keep those.

2. Require date to parse cleanly

You need weekend/weekday.

```
python
 valid_date = (
    pd.to_datetime(
        df["date"],
        errors="coerce"
    )
    .notna()
 )
```

3. Require branch to exist

```
python
 valid_branch = (
    df["branch_code"]
    .isin(branches["branch_code"])
 )
```

4. Require coordinates map to nodes

```python
valid_customer_node = (
    df["customer_node"]
    .notna()
)
```

If lat/lon cannot map:

exclude.

5. Require distance matches graph

Very important.

```python
valid_distance = (
    abs(
        df["calc_distance_km"]
        -
        df["distance_to_customer_KM"]
    ) <= 0.01
)
```

Distance is a model feature, so wrong distance = noisy training.

6. Require loyalty flag valid

Only:

```python
valid_loyalty = (
    df["customerHasloyalty?"]
    .isin([0, 1])
)
```

7. Build trusted subset

```python
trusted_fee_rows = df[
    valid_date
    & valid_branch
    & valid_customer_node
    & valid_distance
```

```
    & valid_loyalty
].copy()
```

Then reverse loyalty and train.

8. Check R²

```
python
model.fit(X, y)

print(
    model.score(X, y)
)
```

Goal:

```
text
> 0.95
```

If lower:

tighten filtering.

9. Use residuals to refine

After first fit:

```
python
trusted_fee_rows["pred"] = (
    model.predict(X)
)

trusted_fee_rows["resid"] = (
    trusted_fee_rows["pred"]
    -
    y
).abs()
```

Drop outliers:

```
python
trusted_fee_rows = (
    trusted_fee_rows[
        trusted_fee_rows["resid"] <= 0.05
```

```
  ]
)
```

Refit.

Often R^2 improves a lot.

A simple mental checklist

A row is reliable if:

```
text
✓ valid date
✓ branch exists
✓ coords map to node
✓ shortest path matches
✓ loyalty is 0/1
✓ fee present
```

Then use it.

Avoid rows where you suspect:

- *branch typo*
- *coordinate mismatch*
- *distance mismatch*
- *ambiguous loyalty*

Because even though there's only one anomaly per row, one wrong feature can skew the regression.

A nice workflow:

```
text
strict subset
→ fit model
→ inspect residuals
→ remove suspicious rows
→ refit
→ use final model to validate remaining rows
```

That tends to work very well for this assignment.

you asked: When shoul

When should I use rule-based imputation vs regression-based imputation?

chatgpt response: For your F

For your FIT5196 task, a good rule is:

> *Use rule-based imputation whenever the value can be derived deterministically. Use regression/statistical imputation only when the value is genuinely missing and depends on patterns.*

*That lines up well with the brief's note that `missing_data.csv` contains **coverage missingness only** (not dirty-data anomalies).*

A practical way to decide:

*1. Use **rule-based imputation** when there is one clear answer*

Choose this if a value can be reconstructed from business rules or other tables.

Typical examples in your assignment:

`order_type`

Missing:

```
text
time = 09:25:10
order_type = NaN
```

Time determines meal exactly:

```
text
Breakfast
```

✅ rule-based

`customer node / lat / lon`

Missing coordinate:

Use `nodes.csv`.

Exact mapping:

```
text
node -> lat/lon
```

✅ rule-based

`distance_to_customer_KM`

Missing:

Compute:

```
text
branch node
→ Dijkstra
→ km
```

One correct answer.

✅ *rule-based*

`order_price`

Missing:

Use inferred item prices:

```
text
sum(qty * unit_price)
```

✅ *rule-based*

`delivery_fee`

Missing:

Use branch linear model:

```
text
base_fee
→ loyalty adjustment
```

Still rule-based because formula is known.

✅

*2. Use **regression-based imputation** when value depends on trends*

Choose this when there's no exact business rule.

Examples:

numeric variable with correlations

If this were missing:

```text
median_income
```

and correlated with:

```text
population
aus_born_perc
```

Then:

```python
LinearRegression()
```

✅ *regression*

fee model training

Even though fee follows rules, coefficients are unknown.

So:

```text
distance + time + weekend
→ fit regression
```

Then fill missing fee.

That's appropriate.

3. Use simple statistical imputation only as fallback

If neither rule nor regression works.

Examples:

```text
median
mode
group-wise median
```

For example:

```python
df.groupby("branch")["x"].transform("median")
```

But for Task 1:

Prefer deterministic logic whenever possible.

Markers usually expect that.

Decision tree

A very useful checklist:

```
text
Missing value?
   ↓
Can another file/business rule determine it exactly?
   YES → rule-based

   NO
   ↓
Strong relationship with other columns?
   YES → regression-based

   NO
   ↓
Use statistical fallback
```

For your specific columns

Here's what I'd use:

Column	*Best method*
`order_type`	*Rule (time)*
`branch_code`	*Rule*
`order_items`	*Rule/menu logic*
`order_price`	*Rule from item prices*
`customer_lat/lon`	*Rule from nodes*
`distance_to_customer_KM`	*Rule via Dijkstra*
`customerHasloyalty?`	*Rule + fee consistency*

delivery_fee	*Regression + loyalty*

Short version:

```
text
Exact business logic exists → rule-based
Need to estimate unknown relationship → regression
No reliable pattern → statistical fallback
```

That's usually the cleanest methodology to explain in your notebook too.

you asked: Which miss

Which missing fields are better recalculated directly instead of predicted?

*chatgpt response: For **`mis*

*For missing_data.csv in this assignment, the best candidates to **recalculate directly** (instead of predicting statistically) are the columns where the brief or supporting files imply **one exact answer**.*

That's usually stronger than prediction and easier to justify in your notebook.

Here's how I'd rank them.

Best to recalculate directly

1. `order_type`

From time.

The brief gives exact windows:

```
text
08:00:00–12:00:00   Breakfast
12:00:01–16:00:00   Lunch
16:00:01–20:00:00   Dinner
```

Example:

```
text
time = 15:42:10
→ Lunch
```

No prediction needed.

✅ direct

2. `customer_lat` / `customer_lon`

From `nodes.csv`.

If customer node or matching coordinates exist:

```
text
node → exact lat/lon
```

Example:

```
text
node 711327755
→ (-37.814..., 144.962...)
```

✅ direct

3. `distance_to_customer_KM`

From graph:

```
text
branch
→ nearest branch node
→ customer node
→ Dijkstra
→ km
```

Exactly reproducible.

✅ direct

4. `order_price`

From item prices.

Once menu prices inferred:

```
text
sum(qty × unit_price)
```

Example:

```
text
2 burgers + 1 fries
= 36.00
```

Deterministic.

✅ direct

5. `branch_code` (sometimes)

If:

- *customer coordinates known*
- *distance known*
- *only one branch produces that distance*

Then:

```
text
branch = NS
```

Can be exact.

✅ often direct

6. `order_items` (sometimes)

If:

- *meal known*
- *price known*
- *item-price combinations unique*

You may reconstruct.

Example:

```
text
Breakfast
price=18
```

Only:

```
text
Coffee + Pancakes
```

fits.

Then exact.

✅ sometimes

Better predicted/modelled

These are usually not directly derivable.

`delivery_fee`

Brief says linear formula differs by branch.

Need:

```
text
 weekend
 time_code
 distance
```

and fit coefficients.

Best via regression.

⚠️ *model*

`customerHasloyalty?`

Sometimes inferable from fee:

```
text
 predicted base fee = 8
 observed = 4
 → loyalty=1
```

But not always obvious.

More model/logic based.

⚠️ *infer*

A useful notebook explanation

You can describe it like this:

```
text
 Fields with deterministic business rules or lookup-table mappings
 (order_type, coordinates, distance, order_price)
 were recalculated directly.

 Fields depending on latent relationships
 (delivery_fee, loyalty in ambiguous cases)
 were estimated using regression/business-rule inference.
```

Short version:

Recalculate directly

✅ order_type ✅ customer_lat ✅ customer_lon ✅ distance_to_customer_KM ✅ order_price ✅ branch_code (when uniquely implied) ✅ order_items (if uniquely implied)

Predict / infer

⚠️ delivery_fee ⚠️ customerHasloyalty?

That tends to produce the cleanest and most defensible imputations.

you asked: What are r

What are reasonable ways to detect delivery_fee outliers?

*chatgpt response: For **`out*

*For outlier_data.csv, the brief says the **only anomaly is outliers in** delivery_fee. That's helpful because every other field can be treated as valid.*

*A good strategy is to combine **business-rule prediction + statistical checks**.*

That's stronger than using IQR alone.

1. Best method: model expected fee, then check residuals

Because the brief tells you fee depends linearly on:

- *weekend / weekday*
- *time of day*
- *distance*
- *branch*
- *loyalty discount*

Build the same fee model you used earlier:

```
text
 one linear regression per branch
```

using:

```
text
 is_weekend
```

```
time_code
distance_to_customer_KM
```

Reverse loyalty before fitting.

Then predict.

Example:

```python
df["pred_fee"] = (
    df.apply(
        predict_fee,
        axis=1
    )
)

df["residual"] = (
    df["delivery_fee"]
    - df["pred_fee"]
).round(2)
```

Then inspect absolute residual:

```python
df["abs_resid"] = (
    df["residual"].abs()
)
```

Rows with large residuals = likely outliers.

This is usually the strongest method.

2. IQR on residuals

Better than IQR on raw fee.

Because fee naturally varies with distance.

Example:

```python
Q1 = df["abs_resid"].quantile(0.25)
Q3 = df["abs_resid"].quantile(0.75)
```

```
IQR = Q3 - Q1

upper = Q3 + 1.5 * IQR
```

Flag:

```python
outliers = df[
    df["abs_resid"] > upper
]
```

Why residuals?

Raw fee:

```text
$3
$12
```

may both be valid depending on distance.

Residual isolates abnormality.

3. Z-score on residuals

Useful second check.

```python
mean = df["residual"].mean()
std = df["residual"].std()

df["z"] = (
    (df["residual"] - mean)
    / std
)
```

Flag:

```python
abs(z) > 3
```

Good sanity check.

4. Visual checks

Very useful in notebook.

Histogram

```python
df["residual"].hist()
```

Expect:

```text
cluster near zero
few far away
```

Boxplot

```python
df.boxplot(column="residual")
```

Clear outliers.

Scatter: predicted vs observed

```python
plt.scatter(
    df["pred_fee"],
    df["delivery_fee"]
)
```

Outliers stand apart.

5. Practical threshold

Often:

```text
abs residual <= 0.5   likely normal
0.5–1.5               inspect
>1.5                  strong outlier
```

But use data distribution, not a hardcoded number.

6. Remove only confident rows

Since the brief says:

> *outlier rows only (no other anomalies)*

You don't correct them.

*You **remove** them.*

Example:

```python
clean = df[
    df["abs_resid"] <= upper
]
```

Recommended workflow

```text
1. reverse loyalty
2. build branch-specific regression
3. predict fee
4. residual = observed - predicted
5. IQR / z-score on residual
6. inspect plots
7. remove flagged rows
```

That's usually the most defensible method in your report because it combines:

- *assignment business logic*
- *statistical evidence*
- *visual EDA*

all together.

you asked: How do I j

How do I justify removing an outlier instead of keeping it as a valid extreme value?

chatgpt response: That's a r

That's a really important part of the notebook write-up.

A good justification is:

> *I removed the row because its delivery_fee is inconsistent with the known business rules and statistically far from the expected range, while all other fields remain valid.*

*That matches the assignment brief: in `outlier_data.csv`, **the only anomalies are outliers in `delivery_fee`**—everything else is assumed valid.*

*A strong explanation usually combines **business logic + statistics + evidence**.*

1. Show it violates the fee model

Example row:

```text
branch = BK
distance = 3.2 km
weekday
Lunch
loyalty = 0

observed fee = 18.50
predicted fee = 6.10
```

Residual:

```text
18.50 - 6.10 = 12.40
```

Justification:

> *Based on the branch-specific linear model, the expected delivery fee was $6.10. The observed fee was $18.50, producing a residual of $12.40, which is inconsistent with the fee-generation rules.*

That's stronger than "looks high."

2. Compare against similar rows

Very persuasive.

Example:

branch	distance	time	fee
BK	3.1	Lunch	5.95
BK	3.2	Lunch	6.12
BK	3.4	Lunch	6.40
BK	3.2	Lunch	**18.50**

Then say:

> *Comparable orders from the same branch and time period fall near $6. The flagged row is substantially outside the observed pattern.*

This helps distinguish a true outlier from a legitimate expensive delivery.

3. Statistical evidence

Example:

Residual IQR:

```text
Q1 = 0.08
Q3 = 0.22
IQR = 0.14


upper bound = 0.43
```

Flagged row:

```text
residual = 12.40
```

Write:

> *The residual exceeded the IQR upper bound by a large margin and was isolated from the rest of the distribution.*

Or z-score:

```text
z = 8.6
```

4. Show it is isolated on a plot

Useful wording:

> *In the residual boxplot and predicted-vs-observed scatter plot, the row appeared clearly separated from the main cluster.*

Even one screenshot in notebook helps.

5. Explain why it's not a valid extreme

This matters.

A valid extreme would still follow the relationship:

```text
longer distance -> higher fee
```

Example:

```
text
distance = 15 km
fee = 16
```

could be valid.

But if:

```
text
distance = 2 km
fee = 18
```

while all nearby rows are $4–7:

not consistent.

Good wording:

> *Although extreme values are not removed automatically, this row was excluded because the fee was not supported by the underlying linear relationship between distance, time and branch, indicating an anomalous observation rather than a legitimate extreme.*

Nice notebook sentence

You can paste something like:

> *Outliers were not removed solely because they were numerically large. A row was removed only when the delivery fee substantially deviated from the branch-specific linear fee model and remained inconsistent compared with neighbouring observations of similar distance and order time. This ensured that legitimate high-fee orders were retained while anomalous delivery-fee records were excluded.*

That's usually a strong justification for marking.

TASK 2

> *From: https://chatgpt.com/c/6a16a287-5680-83ec-9c6a-287f6b29f023*

you asked: "Explain i

"Explain in simple terms what Task 2 is asking me to achieve using the suburb_info.xlsx dataset."

chatgpt response: In simple

In simple terms, Task 2 is asking you to act like a **data analyst preparing data for machine learning**, specifically for a **linear regression model** that predicts house prices.

You are **not required to build the final prediction model**. Instead, your job is to:

1. **Explore the suburb dataset**
2. **Understand how the variables behave**
3. **Check whether the data needs scaling or transformation**
4. **Test different reshaping methods**
5. **Decide which transformations make the data more suitable for linear regression**
6. **Explain and justify your decisions**

The assignment specifically tells you to use only the *suburb_info.xlsx* dataset.

What the dataset contains

The file has suburb-level information such as:

- number of houses
- number of units
- population
- Australian-born percentage
- median income
- median house price

The target variable is:

- *median_house_price*

The other 5 columns are the features used to predict it.

What Task 2 is REALLY about

The core idea is:

> *"How can we reshape the data so linear regression works better?"*

Linear regression works best when:

- variables are on similar scales
- relationships are more linear
- distributions are less skewed
- extreme values are reduced
- variance is more stable

So you are exploring transformations that improve these things.

What you are expected to do step-by-step

Step 1 — Load and understand the dataset

You first inspect:

- shape of data
- data types
- missing values
- summary statistics
- distributions

Typical functions:

```python
df.info()
df.describe()
df.hist()
sns.boxplot()
```

Step 2 — Explore the raw distributions

You check whether variables are:

- highly skewed
- normally distributed
- spread across very different ranges

Example:

Column	Possible Problem
population	very large values
median_income	moderate scale
aus_born_perc	already percentage scale
median_house_price	right-skewed

This helps decide whether reshaping is needed.

Step 3 — Compare scaling methods

The specification explicitly asks you to study methods like:

- Standardisation
- Min-Max Normalisation
- Log transformation
- Power transformation

- Box-Cox transformation

You test how each affects the data.

What each method does (simple explanation)

1. Standardisation

Formula:

Makes variables centered around 0 with standard deviation 1.

Useful when features have different scales.

Example:

- population may be in hundreds of thousands
- aus_born_perc is only between 0–100

2. Min-Max Scaling

Transforms values between 0 and 1.

x' = \frac{x-x_{min}}{x_{max}-x_{min}}

Useful for comparing variables on the same scale.

3. Log Transformation

Usually applied to heavily skewed variables.

Example:

```python
np.log1p(df["population"])
```

It compresses large values and reduces skewness.

4. Power Transformation

Makes distributions more Gaussian/normal.

Good for improving linear relationships.

5. Box-Cox Transformation

Advanced transformation for positive-valued skewed data.

Helps stabilise variance and improve normality.

Step 4 — Check relationship with target variable

This is VERY important.

The assignment says you should evaluate whether transformations improve the relationship with *median_house_price*.

So you should examine things like:

- correlation heatmaps
- scatterplots
- skewness
- linearity
- optional regression diagnostics

Example:

```
python
sns.scatterplot(x=df["median_income"],
                y=df["median_house_price"])
```

You compare:

- before transformation
- after transformation

to see whether the relationship becomes more linear.

Step 5 — Decide the BEST transformation for each column

This is the most important final outcome.

The assignment explicitly says you must give:

> *"a clear, justified final recommended transformation set"*

Meaning you should end with conclusions like:

Column	Recommended Transformation	Why
population	log transform + standardisation	highly skewed
number_of_houses	standardisation	scale differences
aus_born_perc	none	already well scaled
median_income	Box-Cox	improves normality
median_house_price	log transform	reduces skewness

What your notebook should look like

The rubric wants a clear flow:

```
text
Introduction
↓
EDA of raw data
↓
Problems identified
↓
Apply transformations
↓
Compare results
↓
Evaluate suitability for linear regression
↓
Final recommendations
↓
Conclusion
```

VERY IMPORTANT

Task 2 is NOT about:

- getting highest accuracy
- training advanced ML models
- hyperparameter tuning
- feature engineering competitions

It is ONLY about:

> *"Preparing and reshaping the data correctly for linear regression."*

What markers want to see for HD

According to the rubric, HD requires:

- multiple reshaping methods explored
- strong evidence
- detailed explanations
- comparisons

- justified recommendations
- proper narrative and documentation

So your notebook should constantly answer:

- Why was this transformation tried?
- What changed after transformation?
- Did skewness improve?
- Did linearity improve?
- Is scaling necessary?
- Which method is best and why?

That is the real purpose of Task 2.

you asked: What does

What does the assignment mean by preparing the data for a linear regression model rather than building the final model?"

chatgpt response: The assign

The assignment is making an important distinction:

> *Your job is to make the dataset "ready" for linear regression — not to create the best prediction model.*

This means Task 2 focuses on **data preparation and suitability analysis**, not full machine learning modelling.

The specification explicitly says:

> *"The aim of this task is to prepare the data for a linear regression model; it is not to build the final linear model."*

What "preparing the data" means

You are expected to investigate questions like:

- Are the variables on very different scales?
- Are some variables highly skewed?
- Are relationships non-linear?
- Should variables be transformed?
- Which scaling/transformation method works best?
- Does the data better satisfy linear regression assumptions after reshaping?

So Task 2 is mainly:

```text
EDA + Transformation Analysis + Justification
```

not:

```text
Model optimisation + Prediction competition
```

Think of it like this

Imagine building a house.

Linear regression is the house.

Task 2 is preparing the land before construction:

- leveling the ground
- removing obstacles
- checking stability
- preparing foundations

You are not actually building the final house yet.

What they EXPECT you to do

You SHOULD:

1. Explore distributions

Example:

- population may be heavily skewed
- median_income may be moderately skewed

You discuss this using:

- histograms
- boxplots
- skewness values

2. Compare scales

Example:

Feature	**Scale**
aus_born_perc	0–100

population	5,000–200,000
median_income	30,000–120,000

This creates problems for linear regression.

So you test:

- standardisation
- min-max scaling

3. Examine linear relationships

You check whether transformations improve relationships with the target:

```python
sns.scatterplot(x=df["median_income"],
                y=df["median_house_price"])
```

Then maybe compare after log transformation.

4. Evaluate transformation suitability

You compare:

- skewness before/after
- distributions
- correlations
- linearity
- optional R^2 improvements

This is exactly what the rubric asks for.

5. Give recommendations

Example:

Column	**Recommended Action**
population	log + standardise
number_of_units	standardise
aus_born_perc	no transformation
median_house_price	Box-Cox

This recommendation section is the key final output.

What they DO NOT expect

You are NOT expected to:

- build a highly accurate regression model
- tune hyperparameters
- compare ML algorithms
- split train/test extensively
- optimise prediction performance
- report final production metrics

This is NOT a predictive modelling assignment.

Why the assignment says this

Because students often mistakenly jump straight into:

```python
LinearRegression().fit(X, y)
```

and spend all their time improving prediction accuracy.

But the assignment is evaluating:

```text
"How well did you analyse and reshape the data?"
```

not:

```text
"How accurate is your final model?"
```

So what role does linear regression play here?

Linear regression is mainly used as a:

diagnostic tool

You can optionally use:

- R^2
- residual plots
- correlation improvements

to evaluate whether transformations improved the data.

The rubric even says regression diagnostics are only one possible form of evidence.

The core objective of Task 2

The actual goal is:

> *"Find the transformations that make the dataset most suitable for future linear regression modelling."*

That means:

- more linear relationships
- less skewness
- comparable scales
- cleaner distributions
- assumptions better satisfied

In one sentence

Task 2 is essentially:

> *"Investigate, reshape, and justify how the suburb data should be transformed so a future linear regression model would work properly."*

you asked: Which colu

Which columns are considered feature variables and which column is the target variable in this task?"

chatgpt response: In Task 2,

In Task 2, the assignment clearly separates the variables into:

Target Variable (the thing you want to predict)

The target variable is:

Column
median_house_price

This is the output/dependent variable for the future linear regression model.

Feature Variables (inputs used to predict the target)

The assignment specifies these 5 columns as the feature attributes:

Feature Variable
number_of_houses
number_of_units
population
aus_born_perc
median_income

These are the predictor/input/independent variables.

What the assignment explicitly says

The specification states:

> *"...predict the target median_house_price using the five feature attributes above..."*

So:

```text
Features (X):
- number_of_houses
- number_of_units
- population
- aus_born_perc
- median_income

Target (y):
- median_house_price
```

What about the other columns?

The dataset also contains:

Column	Role
suburb	identifier/index
municipality	categorical descriptive column

These are not part of the required 5 feature variables listed in the specification.

You may briefly explore them during EDA, but the assignment specifically focuses on transforming the five numerical feature columns and possibly the target column.

Important conceptual distinction

In linear regression terminology:

Feature variables (X)

These are the inputs:

```
text
X → used to explain/predict house prices
```

Target variable (y)

This is the output:

```
text
y → the value we want to predict
```

So the conceptual relationship is:

```
text
median_house_price
    =
f(number_of_houses,
  number_of_units,
  population,
  aus_born_perc,
  median_income)
```

Why this matters in Task 2

The assignment wants you to study:

1. Whether the feature variables need scaling/transformation
2. Whether the target variable also needs transformation
3. Whether transformations improve linear relationships between X and y

That is why you will frequently analyse:

- feature distributions
- feature scales
- feature vs target scatterplots

- feature-target correlations
- target skewness

during your EDA and reshaping process.

you asked: “Why is mu

“Why is municipality excluded from the transformation analysis?”

chatgpt response: `municipal

municipality is excluded because Task 2 is specifically focused on analysing and transforming the **numerical variables used in the linear regression setup**.

The assignment explicitly states that the model should predict:

- *median_house_price* (target)

using these five feature attributes:

- *number_of_houses*
- *number_of_units*
- *population*
- *aus_born_perc*
- *median_income*

municipality is not included in that required feature list.

The main reason: municipality is categorical, not numerical

municipality contains names/categories such as councils or regions.

Example:

suburb	**municipality**
Clayton	Monash
Carlton	Melbourne
Box Hill	Whitehorse

These are labels/categories, not continuous numeric values.

Why transformations like log or Box-Cox do not apply

Task 2 focuses on transformations such as:

- standardisation
- min-max scaling
- log transformation
- power transformation
- Box-Cox

These methods are designed for **numeric continuous variables**.

For example:

Log transformation

This only works on numbers.

You cannot compute:

```python
log("Monash")
```

because municipality names are text categories.

Why municipality is less relevant to the assignment objective

The assignment is mainly investigating:

1. scale differences
2. skewness
3. linearity with the target
4. suitability for linear regression

These concepts mainly apply to numeric variables.

For example:

Variable	Has Scale?	Has Skewness?	Can Use Box-Cox?
population	Yes	Yes	Yes
median_income	Yes	Yes	Yes
municipality	No	No	No

Could municipality theoretically be used in regression?

Yes — in real machine learning projects, categorical variables are often encoded using methods like:

- one-hot encoding

- label encoding

Example:

municipality	**Encoded**
Monash	1
Melbourne	2

or:

Monash	**Melbourne**	**Whitehorse**
1	0	0

But the assignment does NOT ask you to do categorical encoding.

It specifically restricts Task 2 to transformation analysis on the six numerical columns.

Important assignment interpretation

The wording strongly suggests the teaching team wants you to focus on:

```text
Numerical reshaping techniques
```

rather than:

```text
Feature engineering for categorical variables
```

So including municipality heavily in transformation analysis may:

- distract from the required task
- waste notebook space
- weaken focus

Best practice for your notebook

You can briefly mention:

> *"municipality is a categorical variable and therefore excluded from scaling and transformation analysis, which focuses on numerical variables required for linear regression preparation."*

That is enough.

Simple summary

municipality is excluded because:

1. it is categorical/text data
2. transformation methods like log and Box-Cox only apply to numerical variables
3. the assignment explicitly focuses on the five numerical feature variables
4. the task is about numerical reshaping for linear regression preparation, not categorical encoding

you asked: “How shoul

“How should I begin exploratory data analysis for this dataset before applying any transformations?”

chatgpt response: You should

You should begin the EDA for Task 2 by treating it like an investigation of the dataset **before making any changes**.

Your first goal is to answer:

```
text
“What problems or characteristics exist in the raw data,
and do they justify transformation or scaling?”
```

The assignment explicitly says you should:

> *“first explore the data to see whether scaling or transformation is necessary”*

So your EDA should move logically from:

- understanding the raw dataset → identifying issues → justifying transformations later.

Recommended EDA Structure for Task 2

A strong HD-level notebook usually follows this flow:

```
text
1. Load dataset
2. Initial inspection
3. Data quality checks
4. Statistical summaries
5. Distribution analysis
6. Scale comparison
7. Outlier investigation
8. Feature-target relationship analysis
```

```
9. Correlation analysis
10. Decide whether transformations are needed
```

STEP 1 — Load the dataset

Start simple.

Example:

```
python
import pandas as pd


df = pd.read_excel("suburb_info.xlsx")
df.head()
```

Then explain:

- what the dataset represents
- what each row means
- target variable
- feature variables

STEP 2 — Inspect structure and data types

You need to understand:

- number of rows
- columns
- data types
- missing values

Example:

```
python
df.info()
```

Then discuss:

- numerical columns
- categorical columns
- whether missing values exist

You should mention:

- *municipality* is categorical
- transformation analysis focuses mainly on numerical variables

STEP 3 — Generate descriptive statistics

This is one of the most important starting points.

Example:

```python
df.describe()
```

Now interpret:

- mean
- median
- min/max
- standard deviation

What you should look for

1. Large scale differences

Example:

Variable	**Possible Range**
aus_born_perc	0–100
population	thousands
median_house_price	millions

This immediately suggests scaling may be needed.

2. Potential skewness

If:

- mean >> median
- max extremely high

then the variable may be right-skewed.

This helps justify:

- log transform
- Box-Cox

- power transform

STEP 4 — Visualise distributions

This is critical for Task 2.

Use:

- histograms
- KDE plots
- boxplots

Example:

```python
import matplotlib.pyplot as plt


df.hist(figsize=(12,8))
plt.show()
```

What you should analyse

For EACH numerical variable discuss:

Shape

Is it:

- symmetric?
- right-skewed?
- left-skewed?

Spread

Is variance very large?

Outliers

Are there extreme values?

Example interpretation

Good HD-style explanation:

> *"Population shows a heavily right-skewed distribution with several high-value suburbs creating a long upper tail. This suggests that a logarithmic transformation may improve normality and reduce the influence of extreme observations."*

That kind of interpretation is what markers want.

STEP 5 — Calculate skewness numerically

Very important for justification.

Example:

```python
df.skew(numeric_only=True)
```

Interpretation guide:

Skewness	Meaning
near 0	symmetric
> 1	highly right-skewed
< -1	highly left-skewed

STEP 6 — Compare feature scales

The assignment explicitly says one major goal is:

> *"we want our features to be on the same scale"*

So show this visually.

Example:

```python
df.describe().T[["mean","std","min","max"]]
```

Then explain why scaling may be necessary.

STEP 7 — Explore relationships with the target

This is one of the MOST important sections.

The target is:

```text
median_house_price
```

You should investigate whether relationships are:

- linear

- weak
- curved
- affected by skewness

Best visualisation

Scatterplots.

Example:

```python
import seaborn as sns

sns.scatterplot(
    x=df["median_income"],
    y=df["median_house_price"]
)
```

Do this for each feature.

What to discuss

You should analyse:

- linearity
- clustering
- spread
- curvature
- heteroscedasticity
- influence of extreme points

STEP 8 — Correlation analysis

This helps evaluate linear relationships.

Example:

```python
corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True)
```

Why this matters

The assignment says:

> *"we want our features to have as much linear relationship as possible with the target variable"*

Correlation analysis directly supports this.

STEP 9 — Decide whether transformation is necessary

ONLY after EDA should you begin transformations.

Your EDA should naturally lead to conclusions like:

Observation	**Suggested Action**
heavy skewness	log transform
different scales	standardisation
extreme outliers	power transform
weak linearity	transformation investigation

Important HD-level mindset

Do NOT just produce plots.

For every plot/statistic, answer:

```
text
1. What does this show?
2. Why does it matter for linear regression?
3. Does it suggest transformation/scaling?
```

That interpretation is what separates HD from average submissions.

Best opening structure for your notebook

A very strong start is:

```
text
1. Introduction
2. Dataset Overview
3. Initial Data Inspection
4. Descriptive Statistics
```

```
5. Distribution Analysis
6. Feature Scale Comparison
7. Feature–Target Relationship Analysis
8. Correlation Analysis
9. Summary of Findings Before Transformation
```

Then you move into:

- standardisation
- min-max scaling
- log transforms
- Box-Cox
- power transforms
- comparisons
- final recommendations.

you asked: What descr

What descriptive statistics should I examine first to understand the scale and distribution of the variables?”

chatgpt response: The first

The first descriptive statistics you should examine are the ones that help you understand:

1. **Scale differences**
2. **Spread/variability**
3. **Skewness**
4. **Potential outliers**
5. **Whether transformation may be needed**

For Task 2, the most important statistics come from:

```
python
df.describe()
```

because it gives the core summary statistics for all numerical variables.

The key descriptive statistics you should focus on

1. Mean

The average value.

Example:

```
text
mean(population)
```

Why it matters

The mean helps you:

- understand the central scale
- compare variables
- detect skewness when compared with median

Example interpretation

If:

Variable	Mean
population	85,000
aus_born_perc	62

you immediately see huge scale differences.

This helps justify scaling methods later.

2. Median (50%)

The midpoint of the data.

In *describe()*, this appears as:

```
text
50%
```

Why it matters

Comparing mean vs median helps detect skewness.

Important interpretation rule

Situation	Meaning
mean ≈ median	roughly symmetric

mean > median	right-skewed
mean < median	left-skewed

Example

If:

Statistic	**Value**
Mean income	120,000
Median income	85,000

then the distribution is likely right-skewed.

This may justify:

- log transform
- Box-Cox
- power transform

3. Standard Deviation (std)

Measures spread/variability.

Why it matters

Large standard deviation means:

- values are highly dispersed
- scale may dominate regression
- scaling could help

Example

Variable	**Std**
aus_born_perc	8
population	70,000

This indicates very different scales.

4. Minimum and Maximum

These help identify:

- scale ranges
- extreme values
- possible outliers

Why important for Task 2

The assignment specifically cares about:

- scale comparison
- skewness
- transformation suitability

Large max values often indicate skewness.

Example

Variable	**Min**	**Max**
population	1,200	250,000

This huge spread suggests:

- strong skewness
- possible log transformation need

5. Quartiles (25%, 50%, 75%)

These describe distribution spread.

Why they matter

Quartiles help:

- understand clustering
- identify long tails
- detect outlier behaviour

Example interpretation

If:

Statistic	**Value**

75% population	60,000
Max population	250,000

then only a few suburbs have extremely high population.

This suggests heavy right skew.

6. Range

You can compute:

\text{Range} = x_{max} - x_{min}

Why range matters

Range shows:

- overall spread
- scale differences between variables

Useful when discussing:

- need for normalisation
- scaling methods

7. Skewness (VERY IMPORTANT)

This is one of the most important statistics for Task 2.

Example:

```python
df.skew(numeric_only=True)
```

Why skewness matters

The assignment heavily focuses on transformation methods like:

- log
- Box-Cox
- power transforms

These are mainly used to reduce skewness.

Interpretation guide

Skewness	Meaning
near 0	symmetric
> 1	highly right-skewed
< -1	highly left-skewed

Example

Variable	Skewness
population	2.8

This strongly suggests:

- log transformation may help
- distribution is not ideal for linear regression

8. Correlation with target

Although technically not a descriptive statistic, this is essential early EDA.

Example:

```python
df.corr(numeric_only=True)
```

Why important

The assignment says:

> *features should have "as much linear relationship as possible with the target variable"*

So you should examine:

- which variables correlate strongly with `median_house_price`
- whether transformation may improve linearity

The MOST important statistics for this assignment

If you prioritise, focus on these first:

Priority	Statistic	Purpose
1	mean vs median	skewness

2	std	scale/spread
3	min/max	scale + outliers
4	skewness	transformation need
5	quartiles	distribution shape
6	correlations	linear relationship

Strong HD-level interpretation example

Instead of only writing:

```
text
Population has a high standard deviation.
```

write:

> *"Population exhibits a very large standard deviation and a substantial gap between the median and maximum values, indicating high dispersion and strong right skewness. This suggests that scaling and logarithmic transformation may improve suitability for linear regression."*

That kind of interpretation is what the rubric rewards.

you asked: "How do I

"How do I identify whether variables such as number_of_units or population are heavily right-skewed?"

chatgpt response: You identi

You identify whether variables like *population* or *number_of_units* are heavily right-skewed by combining:

1. **Visual inspection**
2. **Descriptive statistics**
3. **Skewness values**

For Task 2, you should ideally use all three together because the rubric rewards evidence-based analysis.

What "right-skewed" means

A right-skewed distribution means:

- most values are small/moderate
- a few very large values stretch the distribution to the right

Visually:

```text
small values clustered here

long right tail
```

This is very common for:

- population
- income
- house prices

because a few suburbs may be extremely large or expensive.

METHOD 1 — Compare Mean vs Median

This is the simplest starting point.

Use:

```python
df.describe()
```

Interpretation rule

Situation	Interpretation
mean ≈ median	roughly symmetric
mean > median	right-skewed
mean >> median	heavily right-skewed

Example

Suppose `population` gives:

Statistic	Value
Mean	78,000
Median	42,000

Because the mean is much larger than the median:

```text
population is likely heavily right-skewed
```

Why?

Because a few extremely large suburbs pull the average upward.

METHOD 2 — Histogram (MOST IMPORTANT VISUAL)

This is the clearest visual method.

Example:

```python
import matplotlib.pyplot as plt


plt.hist(df["population"], bins=30)
plt.xlabel("Population")
plt.ylabel("Frequency")
plt.title("Population Distribution")
plt.show()
```

What to look for

A right-skewed histogram usually looks like:

```text

```

Meaning:

- many small values
- few extremely large values
- long tail on the right side

Strong interpretation example

> *"The population distribution shows a pronounced right tail, with most suburbs concentrated at lower population levels and a small number of suburbs exhibiting extremely large population values. This indicates strong positive skewness."*

That is HD-level discussion.

METHOD 3 — Boxplot

Very useful for identifying:

- skewness

- extreme values
- long upper tails

Example:

```python
import seaborn as sns


sns.boxplot(x=df["population"])
```

What indicates right skewness?

You may observe:

- many outliers on the right
- a very long upper whisker

Example visual:

```text
|====|——————————○ ○ ○ ○
```

This strongly suggests:

- right skewness
- extreme high values

METHOD 4 — Calculate Numerical Skewness (VERY IMPORTANT)

This is the strongest quantitative evidence.

Example:

```python
df["population"].skew()
```

or:

```python
df.skew(numeric_only=True)
```

How to interpret skewness values

Skewness Value	Interpretation

near 0	symmetric
0.5 to 1	moderately right-skewed
> 1	heavily right-skewed
> 2	extremely right-skewed

Example

If:

```python
population skewness = 2.7
```

then:

```text
population is strongly/heavily right-skewed
```

METHOD 5 — Mean and Max Comparison

Another useful clue.

Example:

Statistic	**Value**
Median	40,000
75%	65,000
Max	300,000

The max is enormously larger than typical values.

This suggests:

- a long right tail
- extreme upper observations
- right skewness

Why skewness matters for Task 2

The assignment specifically asks you to evaluate transformations like:

- log transform

- Box-Cox
- power transform

These transformations are mainly used to reduce right skewness.

What heavily right-skewed variables usually need

If variables like:

- population
- median_income
- median_house_price

are heavily right-skewed, common recommendations are:

Situation	**Typical Transformation**
mild skew	standardisation
strong right skew	log transform
extreme skew	Box-Cox/power transform

Best practice for your notebook

For each variable:

1. Show descriptive stats

```
python
df["population"].describe()
```

2. Show histogram

3. Show skewness value

```
python
df["population"].skew()
```

4. Interpret it

Example:

> *"Population demonstrates substantial positive skewness (skewness = 2.4), supported by the histogram's long right tail and the large difference between the mean and median values. This suggests that a logarithmic or power transformation may improve normality and reduce the influence of extreme values."*

That is exactly the kind of reasoning Task 2 wants.

you asked: “How can h

“How can histograms help me visually determine whether transformation is necessary?”

chatgpt response: Histograms

Histograms help you visually determine whether transformation is necessary because they show the **shape and distribution** of each variable.

For Task 2, histograms are one of the best ways to identify:

- skewness
- long tails
- extreme values
- uneven spread
- non-normal distributions

—all of which affect how suitable the data is for linear regression.

What a histogram shows

A histogram groups values into ranges (“bins”) and shows how frequently values occur.

Example:

```python
plt.hist(df["population"], bins=30)
```

The shape of the bars tells you how the data is distributed.

What you should look for in histograms

1. Symmetric vs skewed distributions

Symmetric distribution

If the histogram looks balanced:

```text

```

then:

- the variable is approximately normal

- transformation may NOT be necessary

2. Right-skewed distributions

If the histogram looks like:

```
text
```

then:

- most values are small/moderate
- a few extremely large values stretch the distribution rightward

This indicates:

- positive/right skewness
- possible need for log or Box-Cox transformation

Why right skewness matters for regression

Linear regression generally works better when variables are:

- more symmetric
- less skewed
- closer to normal distributions

Heavy skewness can cause:

- unstable coefficients
- dominance of extreme values
- poor linear relationships
- heteroscedasticity

So histograms help you visually detect these issues.

Example: population

Suppose the histogram shows:

```
text
Most suburbs:


Few huge suburbs:
                    _____
```

This suggests:

- most suburbs have moderate population
- a few suburbs are extremely large

This is classic right skew.

Interpretation you could write

> *"The histogram for population displays a pronounced right tail, indicating strong positive skewness. Most suburbs cluster at lower population values, while a small number of suburbs exhibit extremely high populations. This suggests that logarithmic or power transformation may improve the distribution for linear regression preparation."*

That is exactly the kind of explanation markers want.

Histograms also help identify scale compression problems

Sometimes the histogram shows:

```
text
```

Meaning:

- almost all values compressed into a tiny region
- a few huge values dominate the scale

This often means:

- scaling may be required
- transformation may improve spread

Comparing histograms BEFORE and AFTER transformation

This is one of the strongest analyses you can do in Task 2.

Before transformation

Example:

- heavily right-skewed histogram

After log transformation

You may observe:

```text
▂▅▇▅▂
```

Much more symmetric.

What this tells you

The transformation:

- reduced skewness
- compressed extreme values
- improved distribution shape
- likely improved suitability for regression

This is the KEY purpose of histogram comparison

You are not just making plots.

You are evaluating:

```text
"Did the transformation improve the data structure?"
```

Typical transformation conclusions from histograms

Histogram Shape	Interpretation	Possible Action
symmetric	already suitable	no transformation
moderate right skew	some skewness	maybe log/power
extreme right tail	strong skewness	log or Box-Cox
compressed values	scale imbalance	scaling
many extreme values	influential outliers	transformation

Best practice for Task 2

For each important variable:

1. Show raw histogram

```python
plt.hist(df["population"])
```

Discuss:

- skewness
- spread
- tails
- concentration

2. Apply transformation

Example:

```python
log_population = np.log1p(df["population"])
```

3. Show transformed histogram

Compare shapes visually.

4. Explain improvement

Example:

> *"After logarithmic transformation, the distribution becomes substantially more symmetric with reduced right-tail dominance, suggesting improved suitability for linear regression modelling."*

Why histograms are especially important in THIS assignment

The rubric rewards:

- exploration
- evidence
- detailed analysis
- justified transformation decisions

Histograms provide direct visual evidence supporting your recommendations.

Simple summary

Histograms help determine whether transformation is necessary because they visually reveal:

- skewness
- long tails
- extreme values
- uneven spread
- non-normal distributions

If the histogram is heavily skewed or dominated by extreme values, transformation methods like:

- log
- power
- Box-Cox

may improve the variable's suitability for linear regression.

you asked: What does

What does skewness measure, and what skewness values indicate a problematic distribution for linear regression?"

chatgpt response: Skewness m

Skewness measures the **asymmetry of a distribution** — in other words, whether the data is balanced around the centre or stretched more toward one side.

For Task 2, skewness is extremely important because it helps determine whether transformations like:

- log
- power
- Box-Cox

are needed before linear regression.

Simple intuition

Imagine a perfectly balanced distribution:

```
text
```

This has:

```
text
skewness ≈ 0
```

because both sides are balanced.

Right-skewed distribution

If the distribution stretches to the right:

```
text
```

then:

- a few very large values exist
- the right tail is longer
- skewness is positive

This is called:

```text
positive skewness
```

Left-skewed distribution

If the distribution stretches to the left:

```text
```


then:

- a few very small values exist
- skewness is negative

This is called:

```text
negative skewness
```

What skewness actually measures mathematically

Conceptually, skewness measures:

```text
“How unevenly the data is distributed around the mean.”
```

A common formula is:

\text{Skewness} = \frac{E[(X-\mu)^3]}{\sigma^3}

You do NOT need to derive this in Task 2, but understanding the interpretation is important.

How to calculate skewness in Python

Example:

```python
df.skew(numeric_only=True)
```

or:

```
python
df["population"].skew()
```

Interpreting skewness values

This is the most important part.

Common interpretation ranges

Skewness Value	**Interpretation**
near 0	approximately symmetric
-0.5 to 0.5	low skewness
0.5 to 1	moderate skewness
> 1	high/heavy skewness
> 2	extremely skewed

For linear regression, what is problematic?

Generally:

Skewness	**Concern Level**
< 0.5	usually fine
0.5–1	may need attention
> 1	often problematic
> 2	strongly problematic

Why high skewness is problematic for linear regression

Linear regression performs better when variables are:

- relatively symmetric
- closer to normal distributions
- free from extreme long tails

Heavy skewness can cause:

- extreme observations dominating the model
- unstable regression coefficients
- non-linear relationships
- heteroscedasticity
- poor residual behaviour

That is why transformations are commonly applied.

Example interpretation

Suppose:

```python
population skewness = 2.7
```

This means:

- the distribution is heavily right-skewed
- a few suburbs have extremely large populations
- transformation is likely needed

HD-level explanation example

> *"Population exhibits substantial positive skewness (2.7), indicating a heavily right-skewed distribution dominated by a small number of highly populated suburbs. Such skewness may reduce the suitability of the variable for linear regression and suggests that logarithmic or Box-Cox transformation should be investigated."*

That is exactly the kind of interpretation the rubric rewards.

Important: skewness alone is NOT enough

You should combine skewness values with:

- histograms
- boxplots
- mean vs median comparisons

because:

- skewness gives numerical evidence
- plots give visual evidence

Together they create a strong justification.

Example workflow for Task 2

Step 1 — Calculate skewness

```python
df.skew(numeric_only=True)
```

Step 2 — Identify problematic variables

Example:

Variable	**Skewness**
population	2.8
median_income	1.4
aus_born_perc	0.2

Step 3 — Decide transformation need

Variable	**Suggested Action**
population	log transform
median_income	power transform
aus_born_perc	probably none

Important nuance for Task 2

Not every skewed variable MUST be transformed.

You should investigate:

- whether skewness meaningfully affects linearity
- whether transformation improves the distribution
- whether transformed variables become more suitable

The assignment wants justified decisions, not automatic transformations.

Simple summary

Skewness measures:

> *how asymmetric a distribution is.*

For linear regression:

- skewness near 0 is generally good
- skewness above 1 is often problematic
- skewness above 2 usually strongly suggests transformation

Heavy skewness indicates that transformations like:

- log
- power
- Box-Cox

may improve the variable's suitability for regression modelling.

you asked: How do QQ

How do QQ plots help assess whether the variables approximately follow a normal distribution?"

chatgpt response: QQ plots (

QQ plots (Quantile–Quantile plots) help you assess whether a variable approximately follows a normal distribution by comparing:

```
text
the actual data distribution
vs
a perfectly normal distribution
```

They are extremely useful in Task 2 because linear regression generally works better when variables are closer to normally distributed, especially after transformation.

What a QQ plot does conceptually

A QQ plot compares:

QQ Plot Axis	Meaning
X-axis	theoretical normal quantiles
Y-axis	actual sample quantiles

If your data follows a normal distribution, the plotted points will fall approximately along a straight diagonal line.

Ideal normal distribution in QQ plot

A good/approximately normal variable looks like:

```text
●
 ●
  ●
   ●
    ●
```

(points closely following a straight line)

This suggests:

- low skewness
- balanced tails
- approximately normal distribution

How to create a QQ plot in Python

Example:

```python
import scipy.stats as stats
import matplotlib.pyplot as plt


stats.probplot(df["population"], dist="norm", plot=plt)
plt.show()
```

How to interpret QQ plots

1. Points close to the line → approximately normal

If most points lie near the straight reference line:

```text
✔ approximately normal
```

Then:

- transformation may not be necessary
- the variable may already be suitable for regression

2. Curved upward pattern → right skewness

A heavily right-skewed variable often produces:

```
text
    ●
   ●
  ●
 ●
   ↑ upper tail bends away
```

Especially at the upper/right end.

This indicates:

- long right tail
- positive skewness
- extreme high values

Common for:

- population
- income
- house prices

3. Curved downward pattern → left skewness

If the left side bends away:

```
text
●
 ●
  ●
   ●
```

with deviation on the lower end, the distribution is negatively skewed.

4. Strong deviations at the ends → heavy tails/outliers

If the middle aligns but ends deviate:

```
text
● ● ● ●
    ●
      ●
```

This suggests:

- outliers
- heavy tails
- non-normal behaviour

Why QQ plots are useful for Task 2

Histograms can sometimes be misleading because:

- shape depends on bin size
- skewness may not be obvious

QQ plots provide a more precise visual check of:

- normality
- tail behaviour
- skewness

This makes them strong evidence when justifying transformations.

Example: population before transformation

Suppose the QQ plot for population shows:

- strong upward deviation at the upper tail
- points far from the line

This suggests:

- heavy right skewness
- non-normal distribution
- possible need for log transformation

After log transformation

You may see:

```
text
points much closer to the line
```

This indicates:

- improved normality
- reduced skewness
- better suitability for linear regression

This is the KEY purpose of QQ plots in Task 2

You are using them to evaluate:

```text
"Did the transformation improve the distribution?"
```

not simply to create visuals.

Example HD-level interpretation

Before transformation:

> *"The QQ plot for population shows substantial deviation from the reference line, particularly in the upper quantiles, indicating strong positive skewness and heavy upper-tail behaviour."*

After log transformation:

> *"Following logarithmic transformation, the QQ plot aligns substantially more closely with the theoretical normal reference line, suggesting improved normality and reduced skewness."*

That is excellent Task 2 analysis.

Important nuance

Variables do NOT need to be perfectly normal.

For Task 2, the goal is:

```text
more suitable for linear regression
```

not mathematical perfection.

So you are mainly checking whether:

- transformations improve symmetry
- tails become less extreme
- distributions become more stable

Best practice for your notebook

For important variables like:

- population
- median_income
- median_house_price

you can show:

1. Histogram
2. Skewness value
3. QQ plot

before and after transformation.

This creates very strong evidence-based analysis, which the rubric rewards.

Simple summary

QQ plots help assess normality by comparing your variable's distribution to a perfect normal distribution.

- points near the line → approximately normal
- upward curvature → right skewness
- downward curvature → left skewness
- large deviations → heavy tails or outliers

In Task 2, QQ plots help you visually evaluate whether transformations like:

- log
- Box-Cox
- power transforms

improve the distribution and make the data more suitable for linear regression.

you asked: How do sca

How do scatter plots help evaluate whether the relationship between a feature and median_house_price is linear?"

chatgpt response: Scatter pl

Scatter plots help you evaluate linearity because they visually show how a feature variable changes in relation to the target variable:

```text
median_house_price
```

For Task 2, scatter plots are one of the most important tools because the assignment explicitly says:

> *features should have "as much linear relationship as possible with the target variable"*

So scatter plots help answer:

```text
“Does this feature relate to house price in a linear way?”
```

What a scatter plot shows

A scatter plot places:

Axis	Variable
X-axis	feature variable
Y-axis	target (`median_house_price`)

Each point represents one suburb.

Example:

```python
sns.scatterplot(
    x=df["median_income"],
    y=df["median_house_price"]
)
```

What “linear relationship” means visually

A linear relationship means the points roughly follow a straight-line trend.

Example of a GOOD linear relationship

```text
        ●
      ●
    ●
  ●
 ●
●
```

This suggests:

- as the feature increases
- house price increases proportionally

This is ideal for linear regression.

Why this matters for regression

Linear regression assumes:

```
text
target changes approximately linearly with features
```

If the scatter plot looks linear:

- regression can model it effectively
- predictions are more reliable
- coefficients become meaningful

What NON-linear relationships look like

1. Curved relationship

Example:

```
text
●
 ●
  ●
   ●
    ●●●
```

This suggests:

- the relationship is not linear
- a straight-line model may perform poorly

Transformation may help.

2. Funnel shape (heteroscedasticity)

Example:

```
text
●
● ●
●  ●
●    ●
```

Spread increases as values grow.

This suggests:

- unstable variance
- possible skewness
- transformation may improve stability

3. Clustered/non-pattern relationship

Example:

```
text
● ● ● ●
 ● ●
●  ● ●
```

No obvious trend.

This indicates:

- weak linear relationship
- feature may not strongly predict house price

How scatter plots help decide transformations

Suppose *population* shows:

- curved relationship
- compressed lower values
- extreme high-value suburbs dominating the plot

This suggests:

- raw variable may not suit linear regression
- log transformation may improve linearity

Example before transformation

Imagine this shape:

```
text
●●●●●●●●
     ●
       ●
         ●
```

This often happens with heavily right-skewed variables.

After log transformation

The relationship may become:

```text
    •
   •
  •
 •
```

Much more linear.

This is the KEY purpose of scatter plots in Task 2

You are evaluating:

```text
"Does transformation improve the linear relationship?"
```

not just creating charts.

Important things to analyse in scatter plots

For each feature, discuss:

What to Observe	Why It Matters
straight-line trend	supports linear regression
curvature	may need transformation
spread changes	variance instability
extreme points	skewness/outlier influence
clustering	weak/nonlinear relationship

Example variables in this dataset

median_income vs median_house_price

You may expect:

- strong positive linear relationship

Higher-income suburbs often have higher house prices.

population vs median_house_price

This may:

- be noisy
- heavily skewed
- non-linear

Transformation may help.

number_of_units vs median_house_price

Could show:

- weak relationship
- clustering
- non-linearity

You analyse whether scaling/transformation improves it.

Strong HD-level interpretation example

> *"The scatter plot between population and median_house_price demonstrates a non-linear pattern with substantial compression among lower population suburbs and several influential high-population observations. This suggests that logarithmic transformation may improve the linear relationship and reduce the dominance of extreme values."*

That kind of explanation is exactly what the rubric rewards.

Best practice for Task 2

For each important feature:

1. Plot raw relationship

```python
sns.scatterplot(
    x=df["population"],
    y=df["median_house_price"]
)
```

2. Discuss:

- linearity
- skewness effects
- outliers

- spread

3. Apply transformation

Example:

```
python
df["log_population"] = np.log1p(df["population"])
```

4. Re-plot

Check whether:

- points align more linearly
- spread stabilises
- curvature reduces

Simple summary

Scatter plots help assess linearity because they visually show whether:

- the feature and target follow a straight-line trend
- the relationship is curved or distorted
- extreme values dominate
- transformation improves the relationship

In Task 2, they are essential for deciding whether transformations make the variables more suitable for linear regression modelling.

you asked: “What is h

“What is heteroscedasticity, and why do fan-shaped scatter plots indicate a problem for linear regression?”

chatgpt response: Heterosced

Heteroscedasticity means that the **spread (variance) of errors is not constant across the data**.

In simpler terms:

```
text
the prediction errors become larger or smaller
as the feature values increase
```

This is important because linear regression assumes:

```text
constant variance of errors
```

which is called:

```text
homoscedasticity
```

When this assumption is violated, we get:

```text
heteroscedasticity
```

Simple intuition

Imagine predicting house prices.

If prediction errors are:

- small for cheap suburbs
- huge for expensive suburbs

then the variance is changing across the dataset.

That is heteroscedasticity.

What it looks like in scatter plots

A common visual sign is a:

```text
fan-shaped pattern
```

Example:

```text
•
• •
•  •
•    •
•      •
```

The spread widens as values increase.

Why this happens

Often:

- smaller feature values are tightly clustered
- larger feature values become increasingly variable

This commonly occurs with:

- population
- income
- house prices

because larger suburbs or wealthier suburbs tend to have much more variation.

Why fan-shaped scatter plots are problematic

Linear regression assumes the residuals/errors have:

```text
constant variance
```

Mathematically:

Var(\varepsilon_i)=\sigma^2

meaning every observation should have roughly equal error variance.

In heteroscedasticity

The variance changes:

```text
small variance here
large variance there
```

So the model fits some regions much better than others.

Why this hurts linear regression

Heteroscedasticity can cause:

Problem	**Effect**
unstable error variance	unreliable predictions
inefficient coefficient estimates	poorer model quality

misleading statistical tests	incorrect p-values/confidence intervals
domination by large-value regions	biased fitting behaviour

Example in this assignment

Suppose you plot:

```
python
sns.scatterplot(
    x=df["median_income"],
    y=df["median_house_price"]
)
```

and observe:

```
text
low income suburbs:
tight clustering

high income suburbs:
very wide spread
```

This indicates:

- variance increases with income
- heteroscedasticity exists

Why transformations help

Transformations like:

- log
- Box-Cox
- power transforms

often compress large values and stabilise variance.

Example:

Before transformation:

```
text
fan shape
```

After log transformation:

```
text
more evenly spread cloud
```

This indicates improved homoscedasticity.

Why this matters for Task 2

The assignment specifically asks you to assess whether transformations improve suitability for linear regression.

Reducing heteroscedasticity is one major reason transformations are applied.

How to identify heteroscedasticity visually

Good linear-regression pattern

You want:

```
text
even spread throughout
```

Example:

```
text
  ● ●
 ● ● ●
● ● ● ●
 ● ● ●
```

Variance remains relatively constant.

Problematic fan-shaped pattern

Example:

```
text
●
● ●
●  ●
●    ●
●      ●
```

Spread widens progressively.

This strongly suggests heteroscedasticity.

HD-level interpretation example

> *"The scatter plot between population and median_house_price exhibits a noticeable fan-shaped pattern, where the variance of house prices increases at higher population levels. This suggests heteroscedasticity, violating the constant variance assumption of linear regression. A logarithmic transformation may stabilise the variance and improve model suitability."*

That is exactly the kind of reasoning Task 2 rewards.

Important nuance

Heteroscedasticity is not always catastrophic.

Task 2 is exploratory, so your goal is mainly to:

```
text
detect it
explain it
investigate whether transformations improve it
```

not mathematically "solve" everything perfectly.

Best practice for your notebook

For important variables:

- show scatter plot before transformation
- discuss spread pattern
- apply transformation
- compare scatter plot after transformation

If the spread becomes more uniform, explain that the transformation improved variance stability.

Simple summary

Heteroscedasticity means:

> *the variance of errors changes across the data.*

Fan-shaped scatter plots indicate this because:

- the spread widens or narrows as values increase
- variance is not constant
- this violates a key linear regression assumption

Transformations like:

- log
- power
- Box-Cox

can often reduce heteroscedasticity and make the data more suitable for linear regression.

you asked: Why is sca

Why is scale disparity between features considered a problem when fitting a regression model?”

chatgpt response: Scale disp

Scale disparity means that different features exist on vastly different numerical ranges.

Example from your dataset:

Feature	**Possible Scale**
aus_born_perc	0–100
population	thousands or hundreds of thousands
median_income	tens of thousands

This creates problems because some variables become numerically dominant during regression modelling.

The assignment explicitly highlights that one important goal is:

> *“we want our features to be on the same scale”*

What “same scale” means

Suppose:

```
text
population = 150,000
aus_born_perc = 62
```

The population values are enormously larger numerically.

Even if both variables are equally important conceptually, the regression algorithm “sees” population values as much larger.

Why this becomes a problem

1. Large-scale variables dominate calculations

Linear regression tries to minimise prediction error mathematically.

Because large-scale variables have much bigger numbers:

- they can disproportionately influence optimisation
- coefficient calculations become unbalanced

Example

Imagine two features:

Feature	**Range**
population	1,000–200,000
aus_born_perc	40–80

Population values are thousands of times larger.

Without scaling:

- population can numerically overpower smaller-scale variables
- smaller variables may appear less influential than they really are

2. Coefficients become difficult to interpret

Suppose the regression equation is:

y = \beta_0 + \beta_1 x_1 + \beta_2 x_2

If variables have wildly different scales:

- coefficients reflect scale differences
- comparisons become misleading

Example:

Feature	**Coefficient**
population	0.003
aus_born_perc	12,000

This does NOT necessarily mean:

- aus_born_perc is more important

The scales distort interpretation.

3. Numerical instability

Large scale differences can create:

- unstable matrix calculations
- rounding issues
- inefficient optimisation

especially when variables differ by several orders of magnitude.

4. Gradient-based optimisation struggles

Many regression-related algorithms (especially ML variants) use gradient descent.

If features are on very different scales:

- optimisation moves unevenly
- convergence becomes slower
- the model may overshoot or converge poorly

Visual intuition

Imagine trying to fit a model where:

```text
population changes by 50,000
aus_born_perc changes by 5
```

The model's optimisation process reacts much more strongly to the larger numerical movements.

Why this matters in Task 2

Task 2 specifically asks you to investigate:

- standardisation
- min-max scaling
- transformation methods

One major reason is to reduce scale disparity.

How scaling solves the problem

Standardisation

Transforms variables to:

- mean = 0
- standard deviation = 1

Now all features exist on comparable scales.

Example after standardisation

Feature	**New Scale**
population	centred around 0
aus_born_perc	centred around 0
median_income	centred around 0

No feature numerically dominates purely because of size.

Min-Max Scaling

Compresses values into:

```
text
0 to 1
```

using:

x' = \frac{x-x_{min}}{x_{max}-x_{min}}

Again, features become comparable.

Important nuance

For ordinary linear regression:

- scale disparity is less catastrophic than for some ML algorithms

BUT it still matters because:

- interpretation improves
- numerical stability improves
- comparisons become fairer
- transformations become easier to evaluate

And the assignment explicitly expects you to analyse this issue.

How to identify scale disparity during EDA

Use:

```python
df.describe().T
```

Look at:

- min
- max
- standard deviation

Example:

Variable	**Max**
population	250,000
aus_born_perc	95

Huge disparity indicates scaling may be necessary.

HD-level interpretation example

> *"Substantial scale disparity exists across the numerical features. Population and median_income operate on much larger numerical ranges compared with aus_born_perc, which may cause certain variables to dominate regression calculations. Standardisation or min-max scaling should therefore be investigated to place features on comparable scales."*

That is the type of explanation the rubric rewards.

Important conceptual point

Scale disparity does NOT mean:

- the larger-scale feature is more important

It only means:

- the numbers themselves are larger

Scaling removes this artificial numerical imbalance.

Simple summary

Scale disparity is problematic because:

- large-scale variables can dominate regression calculations

- coefficients become harder to interpret
- optimisation becomes less stable
- features are not fairly comparable

Scaling methods like:

- standardisation
- min-max normalisation

help place features on comparable numerical ranges, improving suitability for regression modelling.

you asked: Explain ho

Explain how StandardScaler() works mathematically and why it standardises features to mean 0 and standard deviation 1.”

chatgpt response: `StandardS

StandardScaler() is a scaling method that transforms a variable so that:

```
text
mean = 0
standard deviation = 1
```

This process is called:

```
text
standardisation (or z-score normalisation)
```

It is one of the most important preprocessing methods for regression and machine learning.

The mathematical formula

StandardScaler() transforms every value using:

Where:

Symbol	**Meaning**
x	original value
μ	mean of the feature
σ	standard deviation of the feature
z	transformed/standardised value

What the formula is doing conceptually

The formula performs TWO operations:

Step 1 — Subtract the mean

$$ x - \mu $$

This recenters the data around zero.

Example

Suppose:

Population	Mean Population
120,000	80,000

Then:

$$ 120000 - 80000 = 40000 $$

Now the value represents:

- how far above the mean it is

Why this creates mean 0

After subtracting the mean:

- values above mean become positive
- values below mean become negative

The dataset becomes centered around zero.

Simple intuition

Before centering:

```
text
20  30  40  50  60
mean = 40
```

After subtracting mean:

```
text
-20 -10  0  10  20
```

Now the average is:

```
text
0
```

Step 2 — Divide by standard deviation

$$ \frac{x-\mu}{\sigma} $$

This rescales the spread of the data.

Why this matters

Features may have very different spreads.

Example:

Feature	**Std Dev**
population	70,000
aus_born_perc	8

Dividing by the standard deviation normalises the variability.

Why this creates standard deviation 1

Standard deviation measures average spread from the mean.

When every value is divided by the original standard deviation:

- the spread becomes standardised
- one “unit” now means one standard deviation

So the transformed variable has:

```
text
std = 1
```

Intuition of standardised values

After scaling:

Standardised Value	**Meaning**
0	exactly average

+1	1 std above mean
-1	1 std below mean
+2	unusually high
-2	unusually low

Example calculation

Suppose:

Variable	**Value**
population	120,000
mean	80,000
std	20,000

Then:

z = \frac{120000-80000}{20000} = 2

Interpretation:

> *text*
> *this suburb is 2 standard deviations above average population*

Why standardisation is useful

1. Removes scale disparity

Before scaling:

Feature	**Scale**
population	0–200,000
aus_born_perc	0–100

After standardisation:

- both features are centred similarly
- no feature dominates numerically

This directly addresses one of Task 2’s major goals.

2. Makes features comparable

After scaling:

- coefficients become easier to compare
- features contribute more fairly

3. Improves optimisation stability

Many ML/regression algorithms work better when features are similarly scaled.

4. Helps interpret relative extremeness

Standardised values show:

- how unusual observations are
- relative distance from the mean

Important clarification

Standardisation does NOT:

- make data perfectly normal
- remove skewness
- remove outliers

It ONLY:

- centers data
- rescales variance

So if a variable is heavily skewed:

- you may still need log or Box-Cox transformation.

How StandardScaler works in Python

Example:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaled = scaler.fit_transform(df[[
```

```
    "population",
    "median_income"
]])
```

What fit_transform() does

`fit()`

Learns:

- mean
- standard deviation

from the data.

`transform()`

Applies:

to every value.

Important for Task 2

In your notebook, you should explain WHY you are using standardisation.

Good explanation:

> *"Standardisation was applied to reduce scale disparity across features such as population and aus_born_perc. By centering variables around zero and scaling them to unit variance, the transformed features become numerically comparable and more suitable for regression modelling."*

That is the kind of interpretation the rubric rewards.

Simple summary

`StandardScaler()` works by:

1. subtracting the mean
2. dividing by the standard deviation

using:

This transforms features so that:

- mean = 0
- standard deviation = 1

which:

- removes scale disparity
- improves comparability
- stabilises optimisation
- makes the data more suitable for regression modelling.

you asked: What is th

What is the difference between StandardScaler() and MinMaxScaler(), and why do both fail to correct skewness?”

chatgpt response: `StandardS

StandardScaler() and *MinMaxScaler()* are both **scaling methods**, but they work differently mathematically and have different effects on the numerical range of the data.

However, BOTH methods only change:

- the scale
- the range
- the centering

They do **NOT change the actual shape of the distribution**.

That is why they fail to correct skewness.

Core difference

Method	**Main Goal**
StandardScaler()	centre data around 0 with std = 1
MinMaxScaler()	compress values into a fixed range (usually 0–1)

1. StandardScaler()

Mathematical formula:

Where:

- x = original value
- μ = mean
- σ = standard deviation

What StandardScaler does

It:

1. subtracts the mean
2. divides by the standard deviation

Result:

- mean becomes 0
- standard deviation becomes 1

Effect on data

Example:

Original Population	**Standardised**
20,000	-1.2
80,000	0
140,000	+1.5

The relative ordering stays the same.

Only:

- centering
- spread

change.

2. MinMaxScaler()

Mathematical formula:

$$x' = \frac{x-x_{min}}{x_{max}-x_{min}}$$

What MinMaxScaler does

It rescales all values into:

```
text
0 to 1
```

Example

Original Income	**Min-Max Scaled**
30,000	0.0

75,000	0.5
120,000	1.0

Again:

- ordering stays the same
- shape stays the same

Only:

- range changes

Main conceptual difference

StandardScaler	**MinMaxScaler**
centres around mean	rescales to fixed range
values may be negative	values usually 0–1
uses standard deviation	uses min/max
less sensitive to exact bounds	sensitive to outliers

Why BOTH fail to correct skewness

This is the MOST important point.

Skewness is about:

```
text
the SHAPE of the distribution
```

Specifically:

- long tails
- asymmetry
- extreme concentration

Scaling methods DO NOT change shape

Both scalers perform only:

- linear transformations

They stretch or compress values proportionally.

They do NOT:

- bend the distribution
- compress long tails differently
- reduce asymmetry

Important intuition

Imagine a heavily right-skewed distribution:

```text

```

After StandardScaler:

```text
same shape, just recentered/rescaled
```

After MinMaxScaler:

```text
same shape, just squeezed into 0–1
```

The long right tail STILL exists.

Why the skewness remains

Suppose original values are:

```text
1, 2, 3, 4, 100
```

Clearly right-skewed.

After MinMaxScaler

Values become roughly:

```text
0.00, 0.01, 0.02, 0.03, 1.00
```

Still right-skewed.

After StandardScaler

Values become roughly:

```text
-0.5, -0.4, -0.3, -0.2, 2.8
```

Still right-skewed.

The shape NEVER fundamentally changed

Only:

- scale
- centre
- numerical range

changed.

What ACTUALLY fixes skewness

To reduce skewness, you need:

- non-linear transformations

such as:

- log transform
- Box-Cox
- power transforms

because they compress large values more aggressively than small values.

Example: Log transformation

Suppose:

```text
1, 2, 3, 4, 100
```

After log transform:

```text
0, 0.69, 1.10, 1.39, 4.61
```

Now the extreme value is compressed much more.

The shape actually changes.

That is why skewness reduces.

Why this matters for Task 2

Task 2 expects you to distinguish between:

- scaling problems
- skewness problems

These are different issues.

Scaling methods solve:

Problem	**Solved?**
scale disparity	YES
unequal ranges	YES
numerical comparability	YES

But NOT:

Problem	**Solved?**
skewness	NO
long tails	NO
non-normality	NO
heteroscedasticity	usually no

HD-level explanation example

> *"Although StandardScaler and MinMaxScaler successfully place features onto comparable numerical ranges, both methods apply only linear rescaling and therefore preserve the original distribution shape. Consequently, heavily right-skewed variables such as population remain skewed after scaling, indicating that additional transformations such as logarithmic or Box-Cox transformation may still be required."*

That is exactly the kind of interpretation Task 2 rewards.

Simple summary

StandardScaler

Uses:

to create:

- mean = 0
- std = 1

MinMaxScaler

Uses:

x' = \frac{x-x_{min}}{x_{max}-x_{min}}

to rescale values into:

- 0 to 1

Why neither fixes skewness

Because both are:

- linear scaling methods

They change:

- scale
- range
- centering

but NOT:

- distribution shape
- asymmetry
- long tails

So heavily skewed variables remain skewed unless non-linear transformations like:

- log
- power
- Box-Cox

are applied.

you asked: How does l

How does log transformation reduce right-skewness, and why can it sometimes overcorrect a variable like population?”

chatgpt response: Log transf

Log transformation reduces right-skewness because it compresses **large values much more strongly than small values**.

This helps shrink long right tails and makes the distribution more balanced and symmetric — which is often better for linear regression.

However, for variables like `population`, log transformation can sometimes compress the data *too aggressively*, causing:

- loss of meaningful variation
- overcompression of large suburbs
- distortion of relationships
- weaker interpretability

This is what people mean by "overcorrection."

Why right-skewness happens

A right-skewed variable usually looks like:

```text

```

Meaning:

- most values are small/moderate
- a few extremely large values create a long right tail

This is common for:

- population
- income
- house prices

What log transformation does mathematically

The transformation is:

or commonly in Python:

```python
np.log1p(x)
```

which computes:

\log(1+x)

to safely handle zeros.

The key idea: logarithms compress large numbers

This is the MOST important concept.

Example

Suppose population values are:

Original	Log Value
10	2.30
100	4.61
1,000	6.91
10,000	9.21
100,000	11.51

Notice:

```
text
huge original jumps
become much smaller log jumps
```

Why this reduces skewness

In a right-skewed distribution:

- the problem comes from extreme large values

Log transformation compresses those extreme values much more than smaller ones.

So:

- long tails shrink
- outliers become less dominant
- distribution becomes more symmetric

Visual intuition

Before log transform

```
text
```

Strong right tail.

After log transform

```
text
```


Much more balanced.

Why this helps linear regression

Reducing skewness can improve:

- normality
- linearity
- variance stability
- homoscedasticity

This directly supports Task 2's objective of preparing data for linear regression.

Why population often becomes problematic

Population variables frequently contain:

- genuinely meaningful large differences
- natural large-scale structure

For example:

Suburb	**Population**
Small suburb	5,000
Medium suburb	50,000
Large suburb	250,000

These differences may contain important predictive information.

What overcorrection means

Log transformation may compress the upper range so much that:

- large suburbs become too similar numerically
- important variation disappears

Example

Original values:

Population
5,000
50,000
250,000

After log:

Log Population
8.52
10.82
12.43

The huge real-world differences become much closer together.

Why this can be a problem

If house prices truly differ strongly between:

- medium-population suburbs
- mega-population suburbs

then excessive compression may:

- weaken the relationship
- reduce interpretability
- flatten important patterns

Another way to think about it

Log transformation assumes:

```text
relative differences matter more than absolute differences
```

Example:

- increasing from 10k → 20k
- increasing from 100k → 200k

both become similar proportional increases.

But in some datasets, absolute size may still matter strongly.

How overcorrection appears visually

Before transformation

Scatterplot may show:

- some right skew
- but still meaningful spread

After log transformation

You may observe:

- points overly compressed
- reduced separation
- weaker apparent trend

Example HD-level interpretation

> *"Although logarithmic transformation substantially reduced the right-skewness of population, the transformed distribution appeared overly compressed at higher population levels, reducing variation among highly populated suburbs. This suggests that the transformation may partially overcorrect the variable, potentially weakening meaningful structural differences relevant to house-price modelling."*

That is excellent Task 2 reasoning.

Important nuance

Log transformation is NOT automatically "good."

Task 2 wants you to:

- investigate
- compare
- justify

Sometimes:

- raw variable works better
- standardisation alone is sufficient
- Box-Cox gives better balance
- power transform preserves more structure

The assignment rewards evidence-based decisions, not automatic transformations.

Signs of possible overcorrection

After log transformation, check whether:

Sign	**Possible Issue**
histogram becomes left-skewed	overcompression
scatterplot becomes flattened	weakened relationship
variance becomes too compressed	information loss
correlation decreases	transformation hurt linearity

Best practice for Task 2

For variables like population:

1. Show raw histogram/scatterplot

2. Apply log transformation

```python
df["log_population"] = np.log1p(df["population"])
```

3. Compare:

- skewness
- QQ plots
- histograms
- scatterplots
- correlations

4. Decide whether improvement is genuine

This comparison process is the core of Task 2.

Simple summary

Log transformation reduces right-skewness because it compresses large values much more strongly than small values:

This:

- shrinks long right tails
- reduces dominance of extreme values
- improves symmetry

However, for variables like population, log transformation can sometimes overcompress large values, reducing meaningful variation and potentially weakening relationships important for regression modelling.

you asked: Why is squ

Why is square root transformation considered less aggressive than log transformation for moderately skewed variables?”

chatgpt response: Square roo

Square root transformation is considered **less aggressive** than log transformation because it compresses large values more gently.

Both transformations reduce right-skewness, but:

- log transformation compresses large values very strongly
- square root transformation compresses them more moderately

This makes square root transformation useful for:

- mildly/moderately skewed variables
- situations where you want to reduce skewness *without overcompressing the data*

The mathematical difference

Log transformation

Square root transformation

Key intuition

The logarithm grows MUCH more slowly than the square root.

That means:

- log shrinks large numbers more aggressively
- square root preserves more spacing between large values

Example comparison

Suppose we have:

Original Value	**Square Root**	**Log**
1	1.0	0.0
10	3.16	2.30

100	10.0	4.61
1,000	31.6	6.91
10,000	100	9.21

Notice:

```text
log values compress much more aggressively
```

especially at very large values.

Visual intuition

Original right-skewed distribution

```text
```

After square root transform

```text
```

Moderate improvement.

After log transform

```text
```

Much stronger compression.

Why square root is “less aggressive”

Because it:

- reduces skewness
- but preserves more of the original spacing and variation

This is important when:

- the skewness is only moderate
- the original scale still contains meaningful structure

Why this matters for moderately skewed variables

Suppose a variable has:

```
text
skewness ≈ 0.8 or 1.1
```

This is only moderately right-skewed.

Using log transformation may:

- overcompress the upper tail
- flatten relationships too much
- reduce interpretability

Square root transformation often gives a better balance.

Example: population

Suppose population is moderately skewed.

Problem with log transform

Log may:

- make large suburbs too similar numerically
- compress high-end variation excessively

Square root transform instead

Square root:

- reduces skewness somewhat
- but still preserves distinctions between larger suburbs

So it can improve:

- linearity
- variance stability

without losing too much information.

Another way to think about it

Log transformation says:

```text
“large values should be compressed heavily”
```

Square root transformation says:

```text
“large values should be compressed a little”
```

So square root is a more conservative reshaping method.

Why square root is often used for count-type variables

Variables like:

- population
- counts
- units
- frequencies

often respond well to square root transforms because:

- they are non-negative
- moderately skewed
- variance grows with magnitude

Square root stabilises variance without dramatically distorting structure.

Effect on scatter plots

Suppose raw population vs house price shows:

- mild curvature
- slight heteroscedasticity

Square root transformation may:

- gently straighten the relationship
- stabilise variance
- preserve interpretability

while log transformation may:

- flatten too much
- reduce meaningful spread

HD-level interpretation example

> *"Square root transformation reduced the moderate right-skewness of population while preserving greater separation among higher-value suburbs compared with logarithmic transformation. This suggests that square root transformation may provide a more balanced adjustment for variables exhibiting only moderate skewness."*

That is excellent Task 2 reasoning.

Important conceptual comparison

Transformation	**Aggressiveness**	**Best For**
StandardScaler	very low	scale disparity
MinMaxScaler	very low	range scaling
Square Root	moderate	moderate skew
Log	strong	heavy skew
Box-Cox	adaptive/flexible	complex skewness

Why Task 2 cares about this

Task 2 is NOT asking you to blindly apply transformations.

It wants you to:

- compare methods
- evaluate effects
- justify recommendations

Sometimes:

- log is too aggressive
- square root gives better balance
- raw data may even work better

The key is evidence-based reasoning.

Simple summary

Square root transformation is considered less aggressive than log transformation because it compresses large values more gently.

- log transformation strongly shrinks large values
- square root transformation reduces skewness while preserving more variation

This makes square root transformation useful for moderately right-skewed variables where:

- some skewness reduction is needed
- but excessive compression could distort important relationships.

you asked: Why is squ

Why is square root transformation considered less aggressive than log transformation for moderately skewed variables?”

chatgpt response: Square roo

Square root transformation is considered **less aggressive** than log transformation because it compresses large values more gently.

Both transformations reduce right-skewness, but:

- log transformation compresses large values very strongly
- square root transformation compresses them more moderately

This makes square root transformation useful for:

- mildly/moderately skewed variables
- situations where you want to reduce skewness *without overcompressing the data*

The mathematical difference

Log transformation

Square root transformation

Key intuition

The logarithm grows MUCH more slowly than the square root.

That means:

- log shrinks large numbers more aggressively
- square root preserves more spacing between large values

Example comparison

Suppose we have:

Original Value	**Square Root**	**Log**
1	1.0	0.0

10	3.16	2.30
100	10.0	4.61
1,000	31.6	6.91
10,000	100	9.21

Notice:

```text
log values compress much more aggressively
```

especially at very large values.

Visual intuition

Original right-skewed distribution

```text
```


After square root transform

```text
```


Moderate improvement.

After log transform

```text
```


Much stronger compression.

Why square root is “less aggressive”

Because it:

- reduces skewness
- but preserves more of the original spacing and variation

This is important when:

- the skewness is only moderate
- the original scale still contains meaningful structure

Why this matters for moderately skewed variables

Suppose a variable has:

```
text
skewness ≈ 0.8 or 1.1
```

This is only moderately right-skewed.

Using log transformation may:

- overcompress the upper tail
- flatten relationships too much
- reduce interpretability

Square root transformation often gives a better balance.

Example: population

Suppose population is moderately skewed.

Problem with log transform

Log may:

- make large suburbs too similar numerically
- compress high-end variation excessively

Square root transform instead

Square root:

- reduces skewness somewhat
- but still preserves distinctions between larger suburbs

So it can improve:

- linearity
- variance stability

without losing too much information.

Another way to think about it

Log transformation says:

> *text*
> *“large values should be compressed heavily”*

Square root transformation says:

> *text*
> *“large values should be compressed a little”*

So square root is a more conservative reshaping method.

Why square root is often used for count-type variables

Variables like:

- population
- counts
- units
- frequencies

often respond well to square root transforms because:

- they are non-negative
- moderately skewed
- variance grows with magnitude

Square root stabilises variance without dramatically distorting structure.

Effect on scatter plots

Suppose raw population vs house price shows:

- mild curvature
- slight heteroscedasticity

Square root transformation may:

- gently straighten the relationship
- stabilise variance
- preserve interpretability

while log transformation may:

- flatten too much
- reduce meaningful spread

HD-level interpretation example

> *"Square root transformation reduced the moderate right-skewness of population while preserving greater separation among higher-value suburbs compared with logarithmic transformation. This suggests that square root transformation may provide a more balanced adjustment for variables exhibiting only moderate skewness."*

That is excellent Task 2 reasoning.

Important conceptual comparison

Transformation	**Aggressiveness**	**Best For**
StandardScaler	very low	scale disparity
MinMaxScaler	very low	range scaling
Square Root	moderate	moderate skew
Log	strong	heavy skew
Box-Cox	adaptive/flexible	complex skewness

Why Task 2 cares about this

Task 2 is NOT asking you to blindly apply transformations.

It wants you to:

- compare methods
- evaluate effects
- justify recommendations

Sometimes:

- log is too aggressive
- square root gives better balance
- raw data may even work better

The key is evidence-based reasoning.

Simple summary

Square root transformation is considered less aggressive than log transformation because it compresses large values more gently.

- log transformation strongly shrinks large values
- square root transformation reduces skewness while preserving more variation

This makes square root transformation useful for moderately right-skewed variables where:

- some skewness reduction is needed
- but excessive compression could distort important relationships.

you asked: “How do I

“How do I load and clean the suburb_info.xlsx dataset in Python, including converting percentage and currency columns into usable numeric variables?”

chatgpt response: To begin T

To begin Task 2 properly, you should first:

1. load the Excel dataset
2. inspect the columns
3. clean formatting issues
4. convert percentage/currency text columns into numerical values
5. verify all modelling columns are numeric

This is important because regression analysis and transformations only work correctly on numeric variables.

Step 1 — Import libraries

Start with the core libraries:

```python
import pandas as pd
import numpy as np
```

You will later use:

- matplotlib
- seaborn
- sklearn
- scipy

for EDA and transformations.

Step 2 — Load the Excel dataset

Use:

```python
df = pd.read_excel("suburb_info.xlsx")
```

Then inspect the first rows:

```python
df.head()
```

This helps verify:

- column names
- formatting
- possible symbols like `%` or `$`

Step 3 — Inspect structure and data types

Very important.

Run:

```python
df.info()
```

This tells you:

- which columns are numeric
- which columns are stored as strings (`object`)
- whether cleaning is needed

What you may observe

Sometimes Excel columns like:

Column	Possible Problem
aus_born_perc	`"62%"` stored as text
median_income	`"$85,000"` stored as text
median_house_price	`"$1,250,000"` stored as text

If so, these cannot yet be used for:

- scaling
- regression
- skewness calculations
- transformations

They must first be converted to numeric values.

Step 4 — Check column names

Useful for debugging:

```python
print(df.columns)
```

Step 5 — Clean percentage columns

Suppose:

```text
"62%"
```

appears in `aus_born_perc`.

You need to:

1. remove `%`
2. convert to numeric

Method

```python
df["aus_born_perc"] = (
    df["aus_born_perc"]
    .astype(str)
    .str.replace("%", "", regex=False)
    .astype(float)
)
```

Why this works

`.astype(str)`

Ensures values are treated as strings.

`.str.replace("%", "")`

Removes percentage symbols.

Example:

```text
"62%" → "62"
```

`.astype(float)`

Converts cleaned text into numeric values.

Result:

```text
62.0
```

Now the column can be used for:

- EDA
- scaling
- transformations
- regression

Step 6 — Clean currency columns

Suppose values look like:

```text
"$85,000"
```

or:

```text
"$1,250,000"
```

These are text strings.

You must remove:

- `$`
- commas

before conversion.

Example for median_income

```python
df["median_income"] = (
    df["median_income"]
    .astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
```

```
    .astype(float)
)
```

Example transformation

```
text
"$85,000"
```

becomes:

```
text
85000.0
```

Step 7 — Clean median_house_price

Same process:

```
python
df["median_house_price"] = (
    df["median_house_price"]
    .astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(float)
)
```

Step 8 — Verify all numeric columns

After cleaning:

```
python
df.info()
```

You want columns like:

Column	Desired Type
number_of_houses	int/float
number_of_units	int/float
population	int/float

aus_born_perc	float
median_income	float
median_house_price	float

Step 9 — Check for missing values

Always good practice before EDA:

```python
df.isnull().sum()
```

Step 10 — Generate initial statistics

Now the data is ready for analysis:

```python
df.describe()
```

This begins your Task 2 EDA properly.

Important note about percentages

You have TWO valid options.

Option 1 — Keep as 62

```text
62
```

This is perfectly acceptable and usually simpler.

Option 2 — Convert to decimal

```text
0.62
```

using:

```python
df["aus_born_perc"] /= 100
```

Both are mathematically valid.

For Task 2, most students simply keep:

- 62 instead of 0.62

because scaling methods later will normalise it anyway.

Best-practice cleaning workflow for Task 2

```python
# Load dataset
df = pd.read_excel("suburb_info.xlsx")

# Clean percentage column
df["aus_born_perc"] = (
    df["aus_born_perc"]
    .astype(str)
    .str.replace("%", "", regex=False)
    .astype(float)
)

# Clean currency columns
currency_cols = [
    "median_income",
    "median_house_price"
]

for col in currency_cols:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace("$", "", regex=False)
        .str.replace(",", "", regex=False)
        .astype(float)
    )

# Verify types
print(df.info())
```

```
# Descriptive statistics
print(df.describe())
```

Important Task 2 interpretation

Cleaning here is NOT “Task 1 cleaning.”

You are not fixing anomalies.

You are simply converting formatting representations into:

- usable numeric variables
- analysis-ready features

so that:

- scaling
- skewness analysis
- transformations
- regression preparation

can be performed correctly.

HD-level explanation example

> *“Percentage and currency-formatted columns were converted into numeric representations by removing non-numeric symbols such as %, $, and comma separators before casting to floating-point data types. This preprocessing step ensured that all modelling variables were suitable for statistical analysis, transformation, and regression preparation.”*

That is excellent documentation language for Task 2.

you asked: How can I

How can I perform exploratory data analysis on the feature columns using summary statistics, histograms, skewness values, QQ plots, and scatter plots to evaluate whether transformation is necessary?”

chatgpt response: You should

You should perform EDA for the Task 2 feature columns in a **structured analytical workflow**, where each technique answers a specific question about whether the variables are suitable for linear regression.

The goal is NOT just to generate plots.

The goal is to evaluate:

```text
“Does this variable require scaling or transformation
to become more suitable for linear regression?”
```

The assignment explicitly expects you to:

- explore the data
- analyse transformation effects
- justify recommendations

Step-by-step EDA workflow for Task 2

You should analyse these feature columns:

```text
number_of_houses
number_of_units
population
aus_born_perc
median_income
```

and compare them against the target:

```text
median_house_price
```

STEP 1 — Select numerical variables

After cleaning:

```python
features = [
    "number_of_houses",
    "number_of_units",
    "population",
    "aus_born_perc",
    "median_income"
]

target = "median_house_price"
```

STEP 2 — Generate summary statistics

Start with:

```python
df[features + [target]].describe().T
```

What to analyse

1. Scale disparity

Compare:

- min
- max
- std

Example:

Variable	Max
population	250000
aus_born_perc	95

This suggests scaling may be needed.

2. Skewness clues

Compare:

- mean vs median

If:

```text
mean >> median
```

then the variable is likely right-skewed.

HD-level interpretation example

> *"Population exhibits a substantially larger numerical range and standard deviation compared with the other features, suggesting significant scale disparity. Additionally, the large gap between mean and median indicates strong right-skewness."*

STEP 3 — Calculate skewness values

Very important for transformation decisions.

```python
df[features + [target]].skew()
```

Interpretation guide

Skewness	Interpretation
near 0	symmetric
> 0.5	moderate skew
> 1	heavy skew
> 2	extreme skew

What to conclude

If:

- population skewness = 2.5
- median_income skewness = 1.3

then:

- transformation investigation is justified

STEP 4 — Plot histograms

Histograms visually show:

- skewness
- tails
- concentration
- outliers

Example code

```python
import matplotlib.pyplot as plt

df[features + [target]].hist(
    figsize=(14,10),
    bins=30
```

```
)


plt.tight_layout()
plt.show()
```

What to analyse

For EACH variable discuss:

Shape

- symmetric?
- right-skewed?
- left-skewed?

Tails

- long right tail?
- compressed values?

Extreme values

- outlier dominance?

Example interpretation

> *"The histogram for median_income shows a pronounced right tail, indicating positive skewness with a small number of suburbs exhibiting substantially higher income levels."*

STEP 5 — Create QQ plots

QQ plots assess:

- approximate normality
- tail behaviour
- skewness

Example code

```
python
 import scipy.stats as stats


for col in features + [target]:


    plt.figure(figsize=(5,5))
```

```
    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(f"QQ Plot - {col}")
    plt.show()
```

What to analyse

Points near line

→ approximately normal

Upward deviation at upper end

→ right-skewed

Strong tail deviation

→ non-normality/outliers

Example interpretation

> *"The QQ plot for population shows substantial deviation from the theoretical normal line in the upper quantiles, confirming strong positive skewness and heavy upper-tail behaviour."*

STEP 6 — Create scatter plots against target

This is one of the MOST important parts.

The assignment specifically cares about:

```text
linearity with median_house_price
```

Example code

```python
import seaborn as sns


for col in features:
```

```
plt.figure(figsize=(6,4))

sns.scatterplot(
    x=df[col],
    y=df[target]
)

plt.title(f"{col} vs {target}")

plt.show()
```

What to analyse

1. Linear trend

Do points roughly follow a straight line?

2. Curvature

Does the relationship bend?

3. Fan shape

Does variance widen?

Possible heteroscedasticity.

4. Extreme-value dominance

Do a few large observations distort the relationship?

Example interpretation

> *"The scatter plot between population and median_house_price demonstrates noticeable curvature and increasing variance at higher population levels, suggesting both non-linearity and heteroscedasticity."*

STEP 7 — Correlation analysis

Useful for evaluating linear relationships.

Example code

```
python
corr = df[features + [target]].corr()
```

```
plt.figure(figsize=(8,6))

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Matrix")

plt.show()
```

What to analyse

- strong correlations
- weak correlations
- whether transformations may improve relationships

STEP 8 — Summarise findings BEFORE transformations

This section is extremely important.

You should conclude:

Variable	**Observed Issues**	**Suggested Action**
population	heavy skewness, fan shape	log/sqrt
median_income	moderate skew	sqrt/log
aus_born_perc	already symmetric	no transform
number_of_units	scale disparity	standardisation

Example HD-level summary paragraph

> *"EDA indicates that several variables exhibit characteristics that may reduce suitability for linear regression modelling. Population and median_income demonstrate substantial positive skewness, long right tails, and heteroscedastic scatter patterns relative to median_house_price. In contrast, aus_born_perc appears comparatively symmetric and well-behaved. Significant scale disparity is also present across features, supporting investigation of scaling methods such as standardisation and min-max normalisation."*

That is excellent Task 2 analysis.

Why this workflow is strong for the rubric

The rubric rewards:

- multiple forms of evidence
- detailed analysis
- transformation justification
- exploration
- explanation of effects

This workflow gives:

- numerical evidence
- visual evidence
- regression suitability analysis

all together.

Recommended notebook structure

```
text
1. Dataset Overview
2. Initial Data Inspection
3. Summary Statistics
4. Distribution Analysis
   - Histograms
   - Skewness
   - QQ Plots
5. Feature–Target Relationship Analysis
   - Scatter Plots
   - Correlations
6. Pre-Transformation Findings
7. Transformation Experiments
8. Comparison of Methods
9. Final Recommendations
```

That structure is very strong for HD-level Task 2 work.

you asked: “Explain h

“Explain how to use StandardScaler() and MinMaxScaler() in Python, and how I can compare whether they solve only scale disparity or also improve distribution shape.”

chatgpt response: `StandardS

StandardScaler() and *MinMaxScaler()* are two commonly used preprocessing methods in Python for handling **scale disparity** between features.

For Task 2, you should:

1. apply both methods
2. compare their effects
3. determine whether they:
    - only fix scale differences
    - or also improve skewness/distribution shape

A key insight you are expected to discover is:

```text
scaling methods fix scale disparity
BUT usually do NOT fix skewness
```

This is an important Task 2 conclusion.

STEP 1 — Import required libraries

```python
import pandas as pd
import numpy as np

from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler
)
```

STEP 2 — Select feature columns

```python
features = [
    "number_of_houses",
    "number_of_units",
    "population",
```

```
    "aus_born_perc",
    "median_income"
]
```

STEP 3 — Create feature matrix

```python
X = df[features]
```

PART A — Using StandardScaler()

Mathematical idea

`StandardScaler()` applies:

This:

- subtracts the mean
- divides by standard deviation

Result:

- mean = 0
- std = 1

Python implementation

```python
standard_scaler = StandardScaler()


X_standardised = standard_scaler.fit_transform(X)
```

Convert back to DataFrame

Very important for readability.

```python
X_standardised = pd.DataFrame(
    X_standardised,
    columns=features
)
```

Verify scaling worked

```python
X_standardised.describe().T
```

You should observe:

Statistic	Expected
mean	≈ 0
std	≈ 1

PART B — Using MinMaxScaler()

Mathematical idea

`MinMaxScaler()` applies:

x' = \frac{x-x_{min}}{x_{max}-x_{min}}

This rescales values into:

```text
0 to 1
```

Python implementation

```python
minmax_scaler = MinMaxScaler()


X_minmax = minmax_scaler.fit_transform(X)
```

Convert back to DataFrame

```python
X_minmax = pd.DataFrame(
    X_minmax,
    columns=features
)
```

Verify scaling worked

```python
X_minmax.describe().T
```

You should observe:

Statistic	Expected
min	0
max	1

STEP 4 — Compare whether scaling fixed scale disparity

This is the FIRST thing you should analyse.

Compare summary statistics

Original

```python
X.describe().T
```

Standardised

```python
X_standardised.describe().T
```

Min-Max scaled

```python
X_minmax.describe().T
```

What you should observe

Before scaling:

Feature	Range
population	huge
aus_born_perc	small

After scaling:

- ranges become comparable

- scale disparity disappears

HD-level interpretation

> *“Both StandardScaler and MinMaxScaler successfully reduced scale disparity across the features, ensuring that variables with originally large numerical ranges no longer dominate the dataset numerically.”*

STEP 5 — Compare distribution shapes

This is the MOST important analytical step.

You now investigate:

```text
Did scaling ALSO fix skewness?
```

Plot histograms

Original distributions

```python
X.hist(figsize=(12,8), bins=30)
plt.suptitle("Original Features")
plt.show()
```

Standardised distributions

```python
X_standardised.hist(figsize=(12,8), bins=30)
plt.suptitle("StandardScaler Features")
plt.show()
```

Min-Max distributions

```python
X_minmax.hist(figsize=(12,8), bins=30)
plt.suptitle("MinMaxScaler Features")
plt.show()
```

What you should notice

The distributions will look:

```text
almost IDENTICAL in shape
```

because scaling only:

- stretches
- compresses
- recenters

the data linearly.

Very important insight

If `population` was heavily right-skewed before scaling:

```text
it will STILL be right-skewed afterwards
```

STEP 6 — Compare skewness numerically

This is one of the strongest proofs.

Original skewness

```python
X.skew()
```

StandardScaler skewness

```python
X_standardised.skew()
```

MinMaxScaler skewness

```python
X_minmax.skew()
```

What you will observe

Skewness values will be:

```
text
almost unchanged
```

Example:

Variable	Original	Standardised	Min-Max
population	2.5	2.5	2.5

Why?

Because both scalers are:

- linear transformations

They do NOT alter:

- asymmetry
- tails
- shape

STEP 7 — Compare QQ plots

Optional but strong for HD-level evidence.

Example

```
python
import scipy.stats as stats
import matplotlib.pyplot as plt

stats.probplot(
    X["population"],
    dist="norm",
    plot=plt
)

plt.title("Original Population")
plt.show()
```

Repeat for:

- standardised

- min-max scaled

Important observation

QQ plot shapes remain nearly identical.

Meaning:

- normality did not improve significantly
- skewness remains

STEP 8 — Explain the conclusion

This is the KEY Task 2 insight.

Strong HD-level conclusion

> *"Although StandardScaler and MinMaxScaler successfully resolved scale disparity by placing features on comparable numerical ranges, neither method meaningfully altered the underlying distribution shapes. Histograms, skewness values, and QQ plots remained largely unchanged after scaling, indicating that skewness and non-normality persist. This demonstrates that scaling alone is insufficient for correcting distributional issues, and additional non-linear transformations such as logarithmic or Box-Cox transformation may still be necessary."*

That is exactly the kind of interpretation the rubric rewards.

Important conceptual distinction

Problem	**Solved by Scaling?**
scale disparity	YES
unequal ranges	YES
skewness	NO
long tails	NO
non-normality	NO
heteroscedasticity	usually no

Why this matters for Task 2

Task 2 expects you to distinguish between:

- scaling issues
- distribution-shape issues

These are NOT the same thing.

Best workflow for your notebook

```
text
1. Show raw distributions
2. Apply StandardScaler
3. Apply MinMaxScaler
4. Compare summary statistics
5. Compare histograms
6. Compare skewness values
7. Compare QQ plots
8. Conclude:
   scaling fixed scale disparity
   but not skewness
```

That structure is excellent for HD-level Task 2 analysis.

you asked: “Show me h

“Show me how to apply log transformation only to the right-skewed variables and then compare skewness and feature-target relationships before and after transformation.”

chatgpt response: You should

You should apply log transformation **only to variables that are genuinely heavily right-skewed**, not automatically to every feature.

For Task 2, a strong workflow is:

```
text
1. Identify right-skewed variables
2. Apply log transform selectively
3. Compare skewness before/after
4. Compare histograms/QQ plots
5. Compare feature-target scatterplots
6. Decide whether transformation improved suitability
```

This is exactly the kind of evidence-based analysis the rubric rewards.

STEP 1 — Identify right-skewed variables

Calculate skewness:

```python
features = [
    "number_of_houses",
    "number_of_units",
    "population",
    "aus_born_perc",
    "median_income"
]

target = "median_house_price"

df[features + [target]].skew()
```

Example output

Variable	Skewness
number_of_houses	0.3
number_of_units	0.9
population	2.4
aus_born_perc	0.1
median_income	1.5
median_house_price	1.8

Decide which variables need log transformation

Generally:

Skewness	Suggested Action
< 0.5	usually no transform
0.5–1	maybe investigate
> 1	strong candidate for log transform

Example selection

Suppose you choose:

```python
right_skewed = [
    "population",
    "median_income",
    "median_house_price"
]
```

because they are heavily right-skewed.

STEP 2 — Apply log transformation

Use:

```python
for col in right_skewed:

    df[f"log_{col}"] = np.log1p(df[col])
```

Why use `log1p()` instead of `log()`?

Because:

\log(0)

is undefined.

`np.log1p(x)` computes:

\log(1+x)

which safely handles zero values.

STEP 3 — Compare skewness BEFORE and AFTER

This is one of the most important analyses.

Before transformation

```python
print(df[right_skewed].skew())
```

After transformation

```python
log_cols = [f"log_{c}" for c in right_skewed]


print(df[log_cols].skew())
```

Example comparison

Variable	Before	After
population	2.4	0.5
median_income	1.5	0.3
median_house_price	1.8	0.4

What this means

The transformation:

- reduced right-skewness
- improved symmetry
- made distributions more suitable for regression

HD-level interpretation

> *"Logarithmic transformation substantially reduced the positive skewness of population, median_income, and median_house_price, bringing their distributions closer to symmetry and reducing the dominance of extreme high-value observations."*

STEP 4 — Compare histograms

This provides visual evidence.

Before transformation

```python
df[right_skewed].hist(
    figsize=(12,6),
    bins=30
)


plt.suptitle("Before Log Transformation")
```

```
plt.show()
```

After transformation

```
python
df[log_cols].hist(
    figsize=(12,6),
    bins=30
)

plt.suptitle("After Log Transformation")

plt.show()
```

What you should observe

Before

```
text
```


Strong right tails.

After

```
text
```


More balanced/symmetric.

STEP 5 — Compare QQ plots

This helps evaluate normality improvement.

Example

```
python
import scipy.stats as stats

for col in right_skewed:
```

```
    plt.figure(figsize=(5,5))

    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(f"QQ Plot Before - {col}")

    plt.show()
```

After transformation

```
python
 for col in log_cols:

    plt.figure(figsize=(5,5))

    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(f"QQ Plot After - {col}")

    plt.show()
```

What to analyse

You should observe:

- reduced tail deviation
- points closer to the line
- improved approximate normality

STEP 6 — Compare feature-target relationships

This is CRITICAL for Task 2.

The assignment specifically wants you to investigate whether transformations improve relationships with:

```text
median_house_price
```

Example: population vs house price BEFORE

```python
import seaborn as sns

sns.scatterplot(
    x=df["population"],
    y=df["median_house_price"]
)

plt.title(
    "Population vs Median House Price (Before)"
)

plt.show()
```

AFTER log transformation

```python
sns.scatterplot(
    x=df["log_population"],
    y=df["log_median_house_price"]
)

plt.title(
    "Log Population vs Log Median House Price (After)"
)

plt.show()
```

What you should compare

BEFORE

You may observe:

- curvature
- fan shape
- extreme-value dominance

AFTER

You may observe:

- straighter relationship
- reduced heteroscedasticity
- more even spread
- reduced dominance of extreme suburbs

STEP 7 — Compare correlations

Optional but strong evidence.

Before transformation

```python
df[
    ["population", "median_house_price"]
].corr()
```

After transformation

```python
df[
    ["log_population",
     "log_median_house_price"]
].corr()
```

What to check

Did correlation:

- improve?

- remain stable?
- weaken?

This helps evaluate whether the transformation genuinely improved linearity.

Important nuance

Log transformation is NOT automatically better.

Sometimes it:

- improves skewness BUT
- overcompresses relationships

That is why comparison is essential.

Example of possible overcorrection

Suppose after transformation:

- scatterplot becomes too compressed
- correlation decreases

Then you might conclude:

> *"Although logarithmic transformation substantially reduced skewness, it also compressed variation among higher-value suburbs, potentially weakening the linear relationship."*

This is excellent analytical reasoning.

Strong HD-level conclusion example

> *"Selective logarithmic transformation of the heavily right-skewed variables substantially improved distribution symmetry and reduced upper-tail dominance. Histograms and QQ plots demonstrated improved approximate normality, while scatter plots suggested more stable and linear feature-target relationships with reduced heteroscedasticity. However, the transformation also slightly compressed variation among high-value observations, indicating the importance of balancing skewness reduction against preservation of meaningful structure."*

That is excellent Task 2 documentation.

Recommended workflow for your notebook

```text
1. Calculate skewness
2. Identify heavily right-skewed variables
3. Apply log transformation selectively
```

```
4. Compare skewness values
5. Compare histograms
6. Compare QQ plots
7. Compare scatter plots
8. Compare correlations
9. Decide whether transformation improved suitability
```

That is a very strong HD-level Task 2 analysis structure.

you asked: How do I a

How do I apply square root and Box-Cox transformations in Python, and how can I evaluate which transformation improves linear-regression suitability the most?”

chatgpt response: To perform

To perform strong Task 2 analysis, you should compare multiple transformation methods — especially:

- square root transformation
- Box-Cox transformation

—and evaluate which one best improves:

```
text
1. skewness
2. normality
3. linearity
4. variance stability
5. feature-target relationships
```

The assignment is NOT asking you to blindly apply transformations.

It wants you to justify:

> *which transformation makes the data most suitable for linear regression.*

PART 1 — Square Root Transformation

Mathematical idea

Square root transformation applies:

It:

- moderately compresses large values

- reduces right-skewness
- is less aggressive than log transformation

Best for:

- moderately skewed variables

STEP 1 — Identify moderately/right-skewed variables

Example:

```python
df.skew(numeric_only=True)
```

Suppose:

Variable	Skewness
population	2.4
median_income	1.2
number_of_units	0.8

You may choose:

```python
sqrt_cols = [
    "population",
    "median_income",
    "number_of_units"
]
```

STEP 2 — Apply square root transformation

```python
for col in sqrt_cols:

    df[f"sqrt_{col}"] = np.sqrt(df[col])
```

STEP 3 — Compare skewness

Before:

```python
print(df[sqrt_cols].skew())
```

After:

```python
sqrt_new_cols = [
    f"sqrt_{c}" for c in sqrt_cols
]

print(df[sqrt_new_cols].skew())
```

Example result

Variable	Before	After
population	2.4	0.9
median_income	1.2	0.4

Interpretation

Square root transformation:

- reduced skewness
- but preserved more variation than log transformation

PART 2 — Box-Cox Transformation

What Box-Cox does

Box-Cox is a flexible power transformation.

Instead of forcing:

- log
- sqrt

it automatically finds an optimal transformation strength.

Mathematical form

y(\lambda)=\frac{x^\lambda-1}{\lambda}

When:

```text
λ → 0
```

Box-Cox becomes approximately equivalent to log transformation.

Important requirement

Box-Cox ONLY works for:

```text
strictly positive values
```

No zeros or negatives allowed.

STEP 1 — Import Box-Cox

```python
from scipy.stats import boxcox
```

STEP 2 — Apply Box-Cox

Example:

```python
df["boxcox_population"], lambda_pop = boxcox(
    df["population"]
)
```

What this returns

Transformed values

Stored in:

```text
df["boxcox_population"]
```

Optimal lambda

Stored in:

```text
lambda_pop
```

Example

```python
print(lambda_pop)
```

Output:

```text
0.18
```

This means:

- Box-Cox selected a transformation somewhere between:
  - log
  - square root

Apply to multiple variables

```python
boxcox_cols = [
    "population",
    "median_income",
    "median_house_price"
]

boxcox_lambdas = {}

for col in boxcox_cols:

    transformed, lam = boxcox(df[col])

    df[f"boxcox_{col}"] = transformed

    boxcox_lambdas[col] = lam
```

STEP 3 — Compare skewness

```python
boxcox_new_cols = [
    f"boxcox_{c}" for c in boxcox_cols
]

print(df[boxcox_new_cols].skew())
```

Example comparison

Variable	Original	Sqrt	Log	Box-Cox
population	2.4	0.9	0.4	0.2

Interpretation

Box-Cox often:

- reduces skewness the most
- preserves structure better
- improves normality more effectively

PART 3 — Evaluate Which Transformation Is Best

THIS is the most important part of Task 2.

You are not just applying methods.

You are evaluating:

```text
Which transformation improves
linear-regression suitability the most?
```

You should compare FIVE things

1. Skewness reduction

Compare:

```python
comparison = pd.DataFrame({
```

```
    "Original":
        df["population"],

    "Sqrt":
        df["sqrt_population"],

    "Log":
        df["log_population"],

    "BoxCox":
        df["boxcox_population"]

}).skew()

print(comparison)
```

What to check

Closest to:

```
text
0
```

is generally better.

2. Histograms

Plot before/after.

Example

```
python
cols = [
    "population",
    "sqrt_population",
    "log_population",
    "boxcox_population"
]

df[cols].hist(
```

```
    figsize=(12,8),
    bins=30
)

plt.show()
```

What to check

Which transformation gives:

- most symmetric shape
- least extreme tail
- reasonable spread

3. QQ plots

Compare approximate normality.

Example

```
python
 import scipy.stats as stats

for col in cols:

    plt.figure(figsize=(5,5))

    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(col)

    plt.show()
```

What to check

Which transformation:

- aligns closest to the line
- reduces tail deviation most effectively

4. Feature-target scatter plots

VERY important.

Example

```python
sns.scatterplot(
    x=df["boxcox_population"],
    y=df["boxcox_median_house_price"]
)

plt.show()
```

What to check

Does the transformation:

- improve linearity?
- reduce fan-shape variance?
- reduce extreme-value dominance?

5. Correlation improvement

Optional but strong evidence.

Example

```python
df[
    ["population",
     "median_house_price"]
].corr()
```

vs:

```python
df[
    ["boxcox_population",
```

```
    "boxcox_median_house_price"]
].corr()
```

What to check

Did linear relationship:

- improve?
- remain stable?
- weaken?

IMPORTANT: “Best” transformation is NOT always the strongest

Sometimes:

- Box-Cox reduces skewness the most BUT
- overcompresses relationships

or:

- square root preserves interpretability better

The best choice balances:

- symmetry
- linearity
- variance stability
- preservation of meaningful structure

Example HD-level interpretation

> *“Square root transformation moderately reduced skewness while preserving greater separation among larger suburbs. Log transformation further improved symmetry but introduced noticeable compression at the upper range. Box-Cox transformation produced the strongest improvement in normality and QQ-plot alignment while maintaining relatively stable feature-target relationships, suggesting it provides the best overall suitability for linear regression modelling.”*

That is excellent Task 2 reasoning.

Recommended comparison table for your notebook

Transformation	**Skewness Reduction**	**QQ Plot**	**Scatter Plot Linearity**	**Overall Suitability**
None	poor	poor	curved	low

StandardScaler	no change	no change	little change	scale only
MinMaxScaler	no change	no change	little change	scale only
Square Root	moderate	improved	improved	good
Log	strong	strong	sometimes overcompressed	very good
Box-Cox	strongest	strongest	stable	best overall

This kind of structured comparison is extremely strong for HD-level Task 2 work.

you asked: How can I

How can I compare the effects of different transformations using skewness values, correlation analysis, and transformed distribution plots to justify the final recommendation?”

chatgpt response: To justify

To justify your **final transformation recommendation** in Task 2, you should systematically compare how each transformation affects:

```text
1. skewness
2. distribution shape
3. normality
4. feature-target relationships
5. linearity/correlation
```

The key idea is:

> *Don’t just say a transformation “looks better.” Prove it using multiple forms of evidence.*

This is exactly what the rubric rewards.

Overall comparison workflow

For each important variable (e.g. `population`):

```text
Original
↓
StandardScaler
```

```
↓
MinMaxScaler
↓
Square Root
↓
Log
↓
Box-Cox
```

Then compare:

- skewness
- histograms
- QQ plots
- scatter plots
- correlations

and determine which transformation best improves:

- symmetry
- linearity
- variance stability
- regression suitability

STEP 1 — Create transformed versions

Example:

```python
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler
)

from scipy.stats import boxcox

# Original
df["population_original"] = df["population"]

# Square root
```

```
df["population_sqrt"] = np.sqrt(df["population"])

# Log
df["population_log"] = np.log1p(df["population"])

# Box-Cox
df["population_boxcox"], lambda_pop = boxcox(
    df["population"]
)
```

STEP 2 — Compare skewness values

This gives quantitative evidence.

Example

```
python
skewness_comparison = pd.DataFrame({

    "Original":
        [df["population_original"].skew()],

    "Sqrt":
        [df["population_sqrt"].skew()],

    "Log":
        [df["population_log"].skew()],

    "BoxCox":
        [df["population_boxcox"].skew()]

},
index=["Population"])

print(skewness_comparison)
```

Example output

Transformation	Skewness
Original	2.4
Sqrt	0.9
Log	0.4
Box-Cox	0.2

How to interpret

Closer to:

```text
0
```

means:

- more symmetric
- less skewed
- more regression-friendly

Important nuance

Lowest skewness is NOT automatically “best.”

You must also evaluate:

- feature-target relationships
- interpretability
- possible overcompression

STEP 3 — Compare transformed distribution plots

This gives visual evidence.

Histograms

```python
cols = [
    "population_original",
    "population_sqrt",
    "population_log",
```

```
    "population_boxcox"
]

df[cols].hist(
    figsize=(14,8),
    bins=30
)

plt.tight_layout()

plt.show()
```

What to analyse

For each transformation check:

Observation	**Meaning**
reduced long tail	skewness improved
more symmetry	better normality
excessive compression	possible overcorrection
preserved spread	meaningful structure retained

Example interpretation

> *"Square root transformation moderately reduced the right tail while preserving substantial variation. Log and Box-Cox transformations produced substantially more symmetric distributions, though the logarithmic transformation introduced stronger compression at the upper range."*

That is excellent analysis.

STEP 4 — Compare QQ plots

QQ plots evaluate approximate normality.

Example

```
python
 import scipy.stats as stats
```

```
for col in cols:

    plt.figure(figsize=(5,5))

    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(col)

    plt.show()
```

What to analyse

Check:

- closeness to line
- tail deviations
- upper-tail behaviour

Example interpretation

> *"The Box-Cox transformation produced the closest alignment to the theoretical normal line, indicating the strongest improvement in approximate normality."*

STEP 5 — Compare feature-target relationships

This is one of the MOST important sections.

The assignment specifically emphasises:

- linear relationships with the target

Scatter plot comparison

Original

```
python
 sns.scatterplot(
    x=df["population_original"],
```

```
    y=df["median_house_price"]
)

plt.title("Original")
plt.show()
```

Square root

```
python
sns.scatterplot(
    x=df["population_sqrt"],
    y=df["median_house_price"]
)

plt.title("Square Root")
plt.show()
```

Log

```
python
sns.scatterplot(
    x=df["population_log"],
    y=df["median_house_price"]
)

plt.title("Log")
plt.show()
```

Box-Cox

```
python
sns.scatterplot(
    x=df["population_boxcox"],
    y=df["median_house_price"]
)

plt.title("Box-Cox")
plt.show()
```

What to evaluate

Which transformation:

- straightens the relationship?
- reduces fan shape?
- reduces extreme-value dominance?
- preserves useful variation?

Important insight

Sometimes:

- Box-Cox improves symmetry most BUT
- log produces clearer linearity OR
- square root preserves interpretability better

The “best” transformation balances ALL factors.

STEP 6 — Compare correlations

Correlation helps measure linear association numerically.

Example

```python
corr_original = df[
    ["population_original",
     "median_house_price"]
].corr().iloc[0,1]

corr_sqrt = df[
    ["population_sqrt",
     "median_house_price"]
].corr().iloc[0,1]

corr_log = df[
    ["population_log",
     "median_house_price"]
].corr().iloc[0,1]

corr_boxcox = df[
```

```
    ["population_boxcox",
     "median_house_price"]
].corr().iloc[0,1]
```

Create comparison table

```
python
correlation_df = pd.DataFrame({

    "Transformation":
        ["Original",
         "Square Root",
         "Log",
         "Box-Cox"],

    "Correlation":
        [corr_original,
         corr_sqrt,
         corr_log,
         corr_boxcox]

})

print(correlation_df)
```

What to analyse

Higher absolute correlation may indicate:

- stronger linear relationship

BUT:

- do not rely only on correlation

Visual analysis still matters.

STEP 7 — Create a final comparison summary table

This is EXTREMELY strong for HD-level Task 2 work.

Example

Transformation	Skewness	QQ Plot	Scatter Plot	Correlation	Overall
Original	poor	poor	curved	weak	unsuitable
StandardScaler	unchanged	unchanged	unchanged	unchanged	scale only
MinMaxScaler	unchanged	unchanged	unchanged	unchanged	scale only
Square Root	improved	moderate	improved	moderate	good
Log	strong	strong	more linear	stronger	very good
Box-Cox	best	best	stable	strongest	best overall

STEP 8 — Justify the FINAL recommendation

This is the most important writing section.

You should explain:

- WHY one method is preferred
- WHY others are weaker

Example HD-level final justification

> *"Among the evaluated transformations, Box-Cox transformation produced the strongest overall improvement in linear-regression suitability. It reduced skewness most effectively, generated the closest QQ-plot alignment to normality, stabilised variance in the feature-target scatter plots, and maintained strong linear relationships with median_house_price. While logarithmic transformation also substantially improved symmetry, it introduced slightly greater compression among higher-value suburbs. Square root transformation provided moderate improvement but retained noticeable skewness. Therefore, Box-Cox transformation is recommended for heavily skewed variables such as population and median_income."*

That is excellent Task 2 justification.

Important conceptual point

The assignment is NOT asking:

```text
"Which transformation mathematically looks nicest?"
```

It is asking:

```
text
"Which transformation best prepares the data
for linear regression?"
```

So your recommendation should balance:

- symmetry
- linearity
- variance stability
- interpretability
- preservation of structure

—not just skewness alone.

Best-practice notebook structure

```
text
1. Original EDA
2. Scaling Methods
3. Square Root Transformation
4. Log Transformation
5. Box-Cox Transformation
6. Skewness Comparisons
7. Histogram Comparisons
8. QQ Plot Comparisons
9. Scatter Plot Comparisons
10. Correlation Comparisons
11. Final Recommendation Table
12. Conclusion
```

That structure is extremely strong for HD-level Task 2 work.

you asked: How do I c

How do I compare skewness values, distributions, and feature-target relationships before and after transformation to determine which method performs best?"

chatgpt response: To determi

To determine which transformation performs best in Task 2, you should compare the variables **systematically before and after transformation** using three major categories of evidence:

```text
1. Skewness values
2. Distribution shape
3. Feature-target relationships
```

The goal is to answer:

```text
“Did this transformation make the variable
more suitable for linear regression?”
```

A strong Task 2 analysis combines:

- numerical evidence
- visual evidence
- regression suitability analysis

—not just one metric alone.

Overall comparison workflow

For each important variable (e.g. `population`):

```text
Original
↓
Square Root
↓
Log
↓
Box-Cox
```

Then compare:

- skewness
- histograms
- QQ plots
- scatter plots
- correlations

and decide which transformation gives the best overall balance.

STEP 1 — Create transformed variables

Example for *population*:

```python
from scipy.stats import boxcox

# Original
df["population_original"] = df["population"]

# Square root
df["population_sqrt"] = np.sqrt(df["population"])

# Log
df["population_log"] = np.log1p(df["population"])

# Box-Cox
df["population_boxcox"], lambda_pop = boxcox(
    df["population"]
)
```

STEP 2 — Compare skewness values

This provides quantitative evidence.

Example code

```python
skewness_df = pd.DataFrame({

    "Transformation": [
        "Original",
        "Square Root",
        "Log",
        "Box-Cox"
    ],

    "Skewness": [
```

```
        df["population_original"].skew(),
        df["population_sqrt"].skew(),
        df["population_log"].skew(),
        df["population_boxcox"].skew()
    ]
})

print(skewness_df)
```

Example output

Transformation	**Skewness**
Original	2.4
Square Root	0.9
Log	0.4
Box-Cox	0.2

How to interpret

Closer to:

```
text
0
```

means:

- more symmetric
- less skewed
- closer to normality

Important nuance

Lowest skewness does NOT automatically mean “best.”

You must also evaluate:

- linearity

- scatter patterns
- overcompression
- interpretability

STEP 3 — Compare distributions visually

This gives visual evidence of shape improvement.

Histogram comparison

```python
cols = [
    "population_original",
    "population_sqrt",
    "population_log",
    "population_boxcox"
]

df[cols].hist(
    figsize=(14,8),
    bins=30
)

plt.tight_layout()

plt.show()
```

What to analyse

For each transformation check:

Observation	**Meaning**
long right tail remains	skewness still problematic
more symmetry	improved distribution
excessive compression	possible overcorrection
preserved spread	meaningful variation retained

Example interpretation

> *“Square root transformation moderately reduced the long right tail while preserving substantial variation among larger suburbs. Log and Box-Cox transformations produced substantially more symmetric distributions, though the logarithmic transformation compressed the upper range more aggressively.”*

That is excellent Task 2 reasoning.

STEP 4 — Compare QQ plots

QQ plots help evaluate approximate normality.

Example code

```python
import scipy.stats as stats

for col in cols:

    plt.figure(figsize=(5,5))

    stats.probplot(
        df[col],
        dist="norm",
        plot=plt
    )

    plt.title(col)

    plt.show()
```

What to analyse

Which transformation:

- aligns closest to the diagonal line
- reduces tail deviation
- improves normality most effectively

Example interpretation

> *“The Box-Cox transformation produced the closest alignment to the theoretical normal reference line, indicating the strongest improvement in approximate normality.”*

STEP 5 — Compare feature-target relationships

This is one of the MOST important parts of Task 2.

The assignment explicitly emphasises:

- linear relationships with the target

Scatter plot comparison

Original

```python
sns.scatterplot(
    x=df["population_original"],
    y=df["median_house_price"]
)

plt.title("Original")

plt.show()
```

Square root

```python
sns.scatterplot(
    x=df["population_sqrt"],
    y=df["median_house_price"]
)

plt.title("Square Root")

plt.show()
```

Log

```python
sns.scatterplot(
```

```
    x=df["population_log"],
    y=df["median_house_price"]
)

plt.title("Log")

plt.show()
```

Box-Cox

```
python
sns.scatterplot(
    x=df["population_boxcox"],
    y=df["median_house_price"]
)

plt.title("Box-Cox")

plt.show()
```

What to analyse

Check whether the transformation:

- straightens the relationship
- reduces curvature
- reduces fan shape
- stabilises variance
- reduces dominance of extreme values

Important insight

Sometimes:

- Box-Cox gives best symmetry BUT
- log gives cleaner linearity OR
- square root preserves structure better

The best transformation balances ALL these factors.

STEP 6 — Compare correlations

Correlation gives numerical evidence for linearity.

Example

```python
correlation_df = pd.DataFrame({

  "Transformation": [
    "Original",
    "Square Root",
    "Log",
    "Box-Cox"
  ],

  "Correlation": [

    df[
      ["population_original",
       "median_house_price"]
    ].corr().iloc[0,1],

    df[
      ["population_sqrt",
       "median_house_price"]
    ].corr().iloc[0,1],

    df[
      ["population_log",
       "median_house_price"]
    ].corr().iloc[0,1],

    df[
      ["population_boxcox",
       "median_house_price"]
    ].corr().iloc[0,1]
  ]
```

```
})

print(correlation_df)
```

What to interpret

Higher absolute correlation may suggest:

- stronger linear relationship

BUT:

- visual inspection still matters
- correlation alone is not sufficient

STEP 7 — Build a final comparison table

This is extremely strong for HD-level Task 2 work.

Example

Transformation	**Skewness**	**Histogram**	**QQ Plot**	**Scatter Plot**	**Correlation**	**Overall**
Original	poor	right tail	poor	curved	weak	unsuitable
Square Root	moderate	improved	moderate	improved	stronger	good
Log	strong	symmetric	strong	linear	stronger	very good
Box-Cox	best	best	best	stable	strongest	best overall

STEP 8 — Decide which method performs best

Now justify the recommendation using ALL evidence together.

Strong HD-level justification example

> *"Among the evaluated transformations, Box-Cox transformation produced the strongest overall improvement in regression suitability. It reduced skewness most effectively, generated the closest QQ-plot alignment to normality, stabilised the variance in scatter plots, and maintained the strongest linear relationship with median_house_price. Log transformation also substantially improved symmetry but introduced slightly greater upper-range compression, while square root transformation preserved more structure but retained moderate skewness."*

That is excellent Task 2 analysis.

Important conceptual point

The “best” transformation is NOT simply:

- the one with the lowest skewness

It is:

- the one that best balances:
  - symmetry
  - linearity
  - variance stability
  - interpretability
  - preservation of meaningful structure

This distinction is very important for HD-level reasoning.

Best-practice notebook structure

```text
1. Original Variable Analysis
2. Square Root Transformation
3. Log Transformation
4. Box-Cox Transformation
5. Skewness Comparison
6. Histogram Comparison
7. QQ Plot Comparison
8. Scatter Plot Comparison
9. Correlation Comparison
10. Final Recommendation
```

That structure aligns extremely well with the Task 2 rubric expectations.